

Formal Verification of the Number-Theoretic Transform in the HAETAE Post-Quantum Signature Scheme

A Dissertation Submitted in Partial Fulfillment
of the Requirements for the Degree of
Doctor of Philosophy

Author Name

Department of Computer Science and Engineering
Institution Name

Supervised by:

Supervisor Name

Professor, Department of Computer Science and Engineering

2026

This dissertation is submitted under the license of [CC BY 4.0 International](#)

Abstract

The transition to post-quantum cryptography demands not only mathematically sound security arguments but also *machine-verified* correctness proofs for concrete implementations. This dissertation presents a comprehensive formal verification of the Number-Theoretic Transform (NTT) at the core of the HAETAE lattice-based digital signature scheme—a Korean national standard candidate under the KPQC competition.

We establish functional correctness across three vertically composed abstraction layers. At the *implementation layer*, the high-assurance **Jasmin** programming language produces x86-64 assembly that admits direct extraction into **EasyCrypt** modules. At the *algorithmic layer*, an imperative specification in **EasyCrypt** faithfully models the in-place Cooley–Tukey butterfly structure together with Montgomery modular arithmetic over $\mathbb{Z}_q$, $q = 64513$, including tight coefficient-bound invariants across all eight butterfly stages. At the *mathematical layer*, a fully abstract spectral specification expresses the NTT as a Vandermonde-style evaluation at powers of a primitive 512th root of unity in $\mathbb{Z}_q$.

The key technical contributions are:

1. A machine-checked **Montgomery reduction correctness** proof, showing that $\text{montgomery_reduce}(a) \equiv a \cdot R^{-1} \pmod{q}$ for all a in the 16-bit to 32-bit working range, with explicit arithmetic bit-width bounds.
2. A **butterfly-stage induction** connecting the iterative forward NTT loop in `NTT_Fq.ec` to the closed-form evaluation formula in `NTTFullSpec.ec`, parameterised over the 8-level Cooley–Tukey recursion for the negacyclic polynomial ring $\mathbb{Z}_q[X]/(X^{256} + 1)$.
3. An **inverse NTT correctness** proof that tracks the final normalisation by 256^{-1} and the concrete Montgomery output convention, where the exported

inverse transform returns coefficients represented with one factor of R , as expected for `invntt_tomont`.

4. A **Jasmin-to-EasyCrypt refinement** proof in `RefJasminNTT.ec` connecting the extracted Jasmin semantics to the imperative specifications, with a 16-bit-to-24-bit coefficient-bound analysis for the forward transform and a tighter 16-bit preservation proof for the inverse transform.
5. An **end-to-end statement** in `NTTEndToEnd.ec` asserting that the Jasmin exported functions `poly_ntt_jazz` and `poly_invntt_jazz` compute exactly the abstract spectral NTT and its inverse, through two top-level Hoare statements for the forward and inverse exported functions.

The verification identifies a critical structural difference between the HAETAE NTT and the MLKEM NTT: the former performs a *complete* 256-point negacyclic transform with 8-bit bit-reversed twiddle indices, whereas the latter applies an incomplete 128-point transform with a different exponent structure. This difference invalidates a naive reuse of existing MLKEM algebra lemmas and motivates new algebraic infrastructure developed in `NTTFullAlgebra.ec`.

The scalar NTT verification compiles in EasyCrypt version 2024 on an x86-64 Linux platform through the top-level `NTTEndToEnd.ec` file. The checked development totals approximately 8 000 lines of EasyCrypt and 245 lines of Jasmin support code.

Keywords: Number-Theoretic Transform, Formal Verification, EasyCrypt, Jasmin, Post-Quantum Cryptography, HAETAE, Montgomery Arithmetic, Lattice-Based Signatures, Functional Correctness, Negacyclic Convolution.

Acknowledgements

I am deeply grateful to my supervisor, **Supervisor Name**, whose insight into formal methods and post-quantum cryptography shaped every aspect of this work. Their patience in guiding me through the intricacies of **EasyCrypt**’s pRHL calculus and the semantic model of **Jasmin** was invaluable.

I thank the members of the HAETAE design team—in particular the authors of the KPQC submission—for open-sourcing the reference C implementation and for helpful discussions about the mathematical structure of the negacyclic NTT.

I am indebted to the **EasyCrypt** development team for producing a tool that makes large-scale cryptographic proofs tractable, and to the Jasmin compiler team for the extraction infrastructure that forms the bridge between implementation and specification.

Financial support was provided by Funding Agency / Grant Number.

City, Month 2026

Author Name

Contents

Abstract	iii
Acknowledgements	v
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Tools and Methodology	3
1.4 Main Contributions	3
1.5 Structure of the Dissertation	4
2 Background	7
2.1 Lattice-Based Cryptography	7
2.1.1 Module Lattices	7
2.1.2 NTT-Friendly Parameters	7
2.2 Number-Theoretic Transform	8
2.2.1 Definition	8
2.2.2 Cooley–Tukey Butterfly Algorithm	8
2.2.3 Negacyclic NTT vs. Standard NTT	9
2.3 Montgomery Modular Arithmetic	9

2.3.1	REDC Algorithm	9
2.3.2	Montgomery Multiplication	10
2.3.3	Signed Montgomery Reduction	10
2.4	The EasyCrypt Proof Assistant	11
2.4.1	Architecture	11
2.4.2	Hoare Logic in EasyCrypt	11
2.4.3	Module System	11
2.4.4	Theory Cloning	12
2.5	The Jasmin Programming Language	12
2.6	Related Work	13
3	The HAETAE Signature Scheme and Its NTT	15
3.1	Overview of HAETAE	15
3.1.1	Algebraic Setting	15
3.1.2	Role of the NTT	16
3.2	HAETAE NTT Parameters	16
3.2.1	Twiddle Factors	16
3.3	Reference C Implementation	17
3.3.1	Forward NTT	17
3.3.2	Inverse NTT	18
3.3.3	Montgomery Reduction	19
3.3.4	Test Harness	19
3.4	Structural Comparison with MLKEM NTT	20
4	Formal Verification Framework	21
4.1	Overview of the Verification Architecture	21

4.2	Layer 0: Jasmin Source	22
4.3	Layer 1: Extracted EasyCrypt Module	22
4.4	Layer 2: Imperative EasyCrypt Specification	23
4.5	Layer 3: Abstract Spectral Specification	23
4.6	End-to-End Composition	24
4.7	Proof Methodology	25
4.7.1	Proceeding Bottom-Up	25
4.7.2	Hoare Logic for Loop Invariants	25
4.7.3	pRHL for Procedure Equivalence	25
4.7.4	Field-Theory Automation	26
4.8	File Dependency Graph	26
4.9	Relationships Between the EasyCrypt Files	26
5	Formalisation of Field and Ring Arithmetic	31
5.1	The Prime Field F_q	31
5.1.1	Field Definition in <code>Fq.ec</code>	31
5.1.2	Signed vs. Unsigned Representation	32
5.1.3	Bit-Width Bounds	32
5.1.4	Signed Arithmetic Lemmas	33
5.2	The $\mathbb{Z}_q$ Field via Theory Cloning	33
5.2.1	<code>GFq.ec</code> : Abstract Field	33
5.2.2	Primitive Root	34
5.3	The Polynomial Ring R_q	34
5.3.1	<code>Rq.ec</code> : Ring Definition	34
5.3.2	Legacy NTT Operators in <code>Rq.ec</code>	35
5.4	Array Infrastructure	36

5.4.1	<code>Array256.ec</code>	36
5.4.2	Word Array Conversion	36
5.5	Reduction and Auxiliary Operations	37
5.5.1	Conditional Addition (<code>caddq</code>)	37
5.5.2	Freeze (<code>freeze</code>)	37
6	Formalisation of Montgomery Arithmetic	39
6.1	Motivation and Design Choices	39
6.2	Montgomery Parameters	39
6.3	Signed Montgomery Reduction	40
6.3.1	Algorithm	40
6.3.2	Mathematical Analysis	41
6.3.3	EasyCrypt Formalisation	41
6.4	Montgomery Multiplication (<code>fqmul</code>)	42
6.5	Twiddle Factor Multiplication in the NTT Butterfly	43
6.6	Bound Tracking Through NTT Stages	43
7	Imperative NTT Specification	45
7.1	Overview of <code>NTT_Fq.ec</code>	45
7.2	Twiddle Factor Table	45
7.3	Forward NTT Procedure	46
7.3.1	Specification	46
7.3.2	Loop Structure Analysis	47
7.3.3	Loop Invariants	47
7.3.4	Termination	48
7.4	Inverse NTT Procedure	48

7.4.1	Specification	48
7.4.2	Differences from the Forward NTT	50
7.4.3	Final Scaling	50
7.5	Round-Trip Consequence	51
7.6	Twiddle Factor Synchronisation	51
8	Algebraic NTT Infrastructure	53
8.1	Overview of the Algebraic Files	53
8.2	Primitive Root Properties	53
8.2.1	Existence and Uniqueness	53
8.2.2	Orthogonality	54
8.3	Bit-Reversal Permutation	54
8.3.1	Definition	54
8.3.2	Key Properties	55
8.3.3	Bit-Reversal and Twiddle Factor Indexing	55
8.4	Stage Decomposition Theorem	56
8.4.1	Partial NTT	56
8.4.2	Butterfly Stage Lemma	56
8.4.3	Full NTT Correctness	57
8.5	Inverse NTT Algebra	57
8.5.1	Abstract Inverse NTT	57
8.6	The Critical Difference from MLKEM	58
9	Jasmin Implementation	59
9.1	Overview	59
9.2	Montgomery Reduction in Jasmin	60

9.2.1	Source Code	60
9.2.2	Jasmin Semantics	60
9.2.3	Correspondence to C	61
9.3	Field Multiplication	61
9.4	Forward NTT Kernel	61
9.4.1	Original Direct Source Code	61
9.4.2	Structural Correspondence	62
9.4.3	Array Access Pattern	63
9.4.4	Full-Safe Stage-Specialised Forward Kernel	63
9.5	Inverse NTT Kernel	65
9.5.1	Inverse Butterfly Structure	66
9.6	Exported Entry Points	67
9.7	Safety and Constant-Time Verification	69
9.7.1	What Safety Means for This Code	69
9.7.2	Regular Constant-Time Typing	72
9.7.3	Speculative Constant-Time Typing	74
9.7.4	Why Source-Level Checks Apply to Assembly	75
9.7.5	Instantiation for the HAETAE NTT	76
9.7.6	Concrete Checks	77
9.8	Generated Assembly	78
10	Jasmin-to-EasyCrypt Refinement Proof	79
10.1	Overview of the Jasmin Refinement Proofs	79
10.1.1	Full-Safe Constant-Time Bridge	80
10.2	Word-to-Coefficient Correspondence	80
10.2.1	The Linking Predicate	80

10.2.2	Preservation Under Arithmetic	81
10.3	Montgomery Reduction Correspondence	81
10.3.1	Word-Level Reduction	81
10.4	Butterfly Equivalence	82
10.4.1	Individual Butterfly	82
10.5	Loop Equivalence Proofs	83
10.5.1	Inner Loop Equivalence	83
10.5.2	Full NTT Loop Equivalence	83
10.6	Main Refinement Lemmas	84
10.7	Proof Engineering Challenges	84
10.7.1	Signed vs. Unsigned Arithmetic	84
10.7.2	64-bit Product Non-Overflow	85
10.7.3	Twiddle Factor Table Agreement	85
10.7.4	In-Place Update Semantics	85
11	End-to-End Correctness	87
11.1	Overview of <code>NTTEndToEnd.ec</code>	87
11.2	Main Correctness Theorems	87
11.2.1	Forward NTT Correctness	87
11.2.2	Inverse NTT Correctness	88
11.3	Round-Trip Consequence	89
11.4	Proof Composition	89
11.5	Status of the Proof	90
11.5.1	Completed Machine-Checked Steps	90
11.5.2	Machine-Checked Top-Level Compilation	91
11.6	Summary of Verification Metrics	91

12 Conclusion	93
12.1 Summary of Contributions	93
12.2 Limitations and Remaining Scope	95
12.2.1 Scalar NTT Scope	95
12.2.2 Stage-Specialised Extraction Boundary	95
12.2.3 No AVX2 Vectorisation	95
12.2.4 Base Multiplication Not Fully Verified	95
12.2.5 Security Proof Integration	96
12.3 Future Work	96
12.4 Closing Remarks	97
A Visualisation Gallery	99
Appendix: Visualisation Gallery	99
Bibliography	105

List of Figures

4.1	Four-layer verification architecture for the HAETAE NTT. Solid arrows are machine-checked proof steps; dashed arrows indicate the end-to-end composition.	21
4.2	Core EasyCrypt file dependency graph for the HAETAE NTT verification. Arrows point from dependency to dependent; low-level Jasmin array clones and standard-library theories are omitted here and recorded in Table 4.1.	26
A.1	Four-layer verification architecture for the HAETAE NTT. Solid arrows are checked proof steps; dashed arrows show final composition. .	99
A.2	One stage of the Cooley–Tukey butterfly for $n = 8$ (shown for clarity; the actual NTT uses $n = 256$ over 8 stages). Red paths carry the $+t$ update to the upper half; blue paths carry the $-t$ update to the lower half. Each pair of red+blue paths forms one butterfly using twiddle factor $w_k = \zeta_0^{\text{brevs}(k)} \cdot R \bmod q$	100
A.3	Data-flow view of the eight-stage forward NTT. Each column is one outer-loop iteration (ℓ value). Green cells are groups fully processed; white cells are untouched. The checked coefficient invariant starts with a 16-bit input bound and exits with a 24-bit postcondition. The twiddle counter column ($k+$) shows how many twiddle factors have been consumed cumulatively.	100
A.4	Dataflow diagram of the signed Montgomery reduction algorithm. The two 32-bit multiplications (QINV and q) and the arithmetic right shift by 32 are the three core operations. The key divisibility invariant ensures that the right shift is exact (no remainder), giving the congruence $t \equiv aR^{-1} \pmod{q}$	101

A.5	The negacyclic polynomial ring R_q and its isomorphism to the NTT evaluation domain. The NTT evaluates the polynomial at all 256 primitive 512th roots of unity (red dots on the unit circle). This is an instance of the Chinese Remainder Theorem decomposition of R_q over $\mathbb{F}_q$	101
A.6	The bit-reversal permutation brev_8 for the first 16 table values $k = 0, \dots, 15$. Red cells in the bit-pattern boxes highlight the set bits. The bottom row gives the concrete signed Montgomery-form table entry <code>jzetas[k]</code> ; index 0 is padding, and the forward NTT consumes indices 1 through 255.	102
A.7	Structure of the EasyCrypt proof judgements. The topmost judgement (pRHL) relates machine-word arrays to field-coefficient arrays with signed bit-width bounds. The middle (Hoare) judgement captures the loop semantics over field arrays. The algebraic equality connects the loop output to the closed-form spectral sum. The bottom (end-to-end) judgement composes all three.	102
A.8	Structural comparison of the HAETAE and MLKEM NTT. Green box: aspects where methodology is transferable between the two verification projects. Red box: the critical structural difference that prevents direct reuse of MLKEM algebra lemmas.	103
A.9	Bound propagation for the checked forward and inverse NTT theorems.	103
A.10	Detailed EasyCrypt module dependency graph. Arrows point from dependency to dependent file; standard-library and some transitive edges are omitted for readability. Nodes are colour-coded by layer: orange = foundations, yellow = imperative specification, green = algebraic specification, red = refinement, cyan = end-to-end.	104

List of Tables

3.1	HAETAE NTT parameters	16
3.2	First sixteen forward twiddle factors <code>zetas[1..16]</code> (signed Montgomery form)	17
3.3	Comparison of HAETAE and MLKEM NTT structures	20
4.1	Relationships between the EasyCrypt source files	27
6.1	Coefficient bounds at each layer of the verification	44
11.1	Verification metrics by file	92

Listings

2.1	Jasmin function signature example	12
3.1	Forward NTT from <code>ntt.c</code>	17
3.2	Inverse NTT from <code>ntt.c</code>	18
3.3	Montgomery reduction from <code>reduce.c</code>	19
4.1	Fragment of extracted procedure in <code>Hpoly_extract.ec</code>	22
4.2	End-to-end theorem statement in <code>NTTEndToEnd.ec</code>	24
5.1	Field declaration in <code>Fq.ec</code>	31
5.2	<code>GFq.ec</code> : Galois field instantiation	33
5.3	Primitive root declaration in <code>NTTFullAlgebra.ec</code>	34
5.4	Ring definition in <code>Rq.ec</code>	34
5.5	Legacy NTT equations in <code>Rq.ec</code>	35
6.1	Signed Montgomery reduction (C reference)	40
6.2	Signed REDC correctness in <code>Montgomery.ec</code>	41
6.3	<code>fqmul</code> specification	42
7.1	Forward NTT specification in <code>NTT_Fq.ec</code>	46
7.2	Inverse NTT specification in <code>NTT_Fq.ec</code>	49
8.1	Bit-reversal definition in <code>NTTFullAlgebra.ec</code>	54
9.1	Montgomery reduction in <code>reduce.jinc</code>	60
9.2	Field multiplication in <code>reduce.jinc</code>	61

9.3	Original direct forward NTT kernel in <code>poly.jinc</code>	61
9.4	Current full-safe forward NTT stage shape in <code>poly.jinc</code>	63
9.5	Original direct inverse NTT kernel in <code>poly.jinc</code> (condensed)	65
9.6	Current full-safe inverse NTT schedule in <code>poly.jinc</code>	66
9.7	Original exported functions in <code>hpoly.jazz</code>	67
9.8	Current constant-time exported wrappers in <code>hpoly.jazz</code>	67
9.9	Compiler checks for the full-safe constant-time implementation	77
10.1	Array correspondence predicate in <code>RefJasminNTT.ec</code>	80
10.2	Main refinement lemmas in <code>RefJasminNTT.ec</code>	84
11.1	Module imports in <code>NTTEndToEnd.ec</code>	87
11.2	End-to-end NTT correctness statement	87
11.3	End-to-end inverse NTT correctness statement	88
11.4	Proof composition in <code>NTTEndToEnd.ec</code>	90

Chapter 1

Introduction

1.1 Motivation

The threat posed by large-scale quantum computers to today’s public-key infrastructure is no longer a theoretical curiosity. Shor’s algorithm reduces the security of RSA, DSA, and elliptic-curve schemes to polynomial time once a sufficiently powerful quantum computer is available [Sho94]. In response, the U.S. National Institute of Standards and Technology (NIST) ran a multi-year post-quantum cryptography standardisation project, selecting four algorithms for standardisation in 2022–2024 [Nat22a], while the Korean Cryptography Module Validation Programme (KPQC) is pursuing its own national standards [Nat22b].

HAETAE [HAE22] is one of the KPQC signature finalists. It is a lattice-based scheme whose security reduces to the Decisional Module Short Integer Solution (DMSIS) hardness assumption over module lattices. Like all modern lattice-based schemes, HAETAE relies on efficient polynomial multiplication modulo $X^{256} + 1$ in the ring $\mathbb{Z}_q[X]/(X^{256} + 1)$, which in practice is implemented via the *Number-Theoretic Transform* (NTT). The NTT reduces an $O(n^2)$ schoolbook convolution to $O(n \log n)$ operations and is the dominant computational primitive in the scheme.

For a cryptographic standard to be trustworthy, correctness proofs cannot stop at pen-and-paper mathematics. History has shown that hand-verified implementations harbour subtle bugs: off-by-one errors in loop bounds, incorrect twiddle factor tables, wrong modular arithmetic, or mishandled Montgomery scaling can silently produce wrong outputs while passing standard test vectors [BL17]. This dissertation takes the position that *machine-checked* formal proofs, linking every line of executable

code to its abstract mathematical specification, are the appropriate gold standard for cryptographic primitives in post-quantum standards.

1.2 Problem Statement

We address the following problem:

*Prove, in a machine-checkable formal system, that the **Jasmin** implementation of the NTT and inverse-NTT in HAETAE is functionally correct with respect to the abstract mathematical specification of the negacyclic NTT over $\mathbb{Z}_q[X]/(X^{256} + 1)$.*

The challenge is to bridge three distinct abstraction levels:

1. **Low-level word arithmetic.** The **Jasmin** source is compiled to x86-64 assembly and works with 32-bit and 64-bit machine words. Correctness at this level means proving arithmetic identities modulo 2^{32} and establishing bounds (e.g., that an intermediate product fits in a 64-bit register without overflow).
2. **Algorithmic (loop) level.** The Cooley–Tukey butterfly algorithm is expressed as a nested three-loop structure with eight stages. Correctness here means showing that after all loops terminate, the array holds the intended Fourier coefficients, using loop invariants expressed in terms of partial NTT computations.
3. **Mathematical (spectral) level.** The NTT is a linear map $\mathbb{Z}_q^{256} \rightarrow \mathbb{Z}_q^{256}$ defined by evaluation at specific powers of a primitive 512th root of unity. Correctness here means proving that the algorithmic output equals the evaluation formula, and that the inverse NTT is its exact algebraic inverse.

Each layer requires different mathematical machinery: modular arithmetic and bit-vector identities at level 1, loop invariants and array relational assertions at level 2, and field theory with primitive roots at level 3. Composing all three in a single proof assistant is the central technical challenge.

1.3 Tools and Methodology

We use two mature high-assurance tools:

Jasmin [Alm+17]. A domain-specific programming language and compiler designed for high-assurance cryptographic implementations. **Jasmin** programs are compiled to provably correct x86-64 assembly and simultaneously admit extraction into the **EasyCrypt** proof assistant as first-class program modules. The language provides explicit register and memory annotations that remove ambiguity from the compiler’s optimisation decisions.

EasyCrypt [BGB09; Bar+13]. A proof assistant built around the probabilistic Relational Hoare Logic (pRHL) calculus, designed specifically for reasoning about cryptographic constructions. An **EasyCrypt** *program logic* allows Hoare-style reasoning over both deterministic and probabilistic procedures. The ambient logic is a dependent type theory supporting classical higher-order reasoning, enabling rich algebraic developments (ring theory, field theory, group theory) within the same tool.

Our proof strategy follows the *refinement* paradigm:

$$\begin{array}{l} \text{Jasmin} \xrightarrow{\text{extraction}} \text{EC module} \\ \xrightarrow{\text{pRHL equivalence}} \text{EC imperative spec} \\ \xrightarrow{\text{loop invariant}} \text{EC abstract spec.} \end{array}$$

Each arrow represents a machine-checked proof step; the composition yields the end-to-end theorem stated in `NTTEndToEnd.ec`.

1.4 Main Contributions

C1 – Montgomery correctness. A machine-checked proof of the Montgomery reducer: it computes $a \cdot R^{-1} \bmod q$ for all inputs in the relevant working range, with $R = 2^{32}$, $q = 64513$, verified bit-width bounds, and signed-integer arithmetic lemmas absent from the **EasyCrypt** standard library (Chapter 6).

C2 – Imperative NTT specification. A fully proved **EasyCrypt** model of the forward and inverse NTT loops in `NTT_Fq.ec`, including twiddle-factor correctness,

coefficient-bound invariants (16-bit input $\rightarrow$ 24-bit output for forward NTT; 16-bit preservation for inverse NTT), and termination witnesses (Chapter 7).

- C3 – Abstract spectral specification.** An algebraic library establishing properties of the primitive root $\zeta_0 = 426$ in $\mathbb{Z}_q$, bit-reversal permutation lemmas, and a butterfly-stage induction connecting the iterative algorithm to the closed-form evaluation formula (Chapter 8).
- C4 – Jasmin refinement.** A pRHL proof in `RefJasminNTT.ec` showing that the extracted `Jasmin` procedures are observationally equivalent to the imperative specifications up to array representation (Chapter 10).
- C5 – End-to-end correctness.** Top-level forward and inverse correctness theorems relate the exported `Jasmin` procedures to their abstract specifications (Chapter 11).

1.5 Structure of the Dissertation

The remainder of this dissertation is organised as follows.

Chapter 2 reviews the relevant background: lattice cryptography, the NTT, post-quantum signature schemes, and the theoretical foundations of formal verification with `EasyCrypt` and `Jasmin`.

Chapter 3 describes the HAETAE scheme, its NTT parameters, and the reference C implementation that serves as the algorithmic ground truth.

Chapter 4 presents the overall formal framework: the three-layer abstraction stack and the proof methodology.

Chapter 5 covers the formalization of the field $\mathbb{Z}_q$ and the polynomial ring $\mathbb{Z}_q[X]/(X^{256} + 1)$ in `EasyCrypt`.

Chapter 6 details the Montgomery reduction formalization and its correctness proof.

Chapter 7 presents the imperative NTT specification in `NTT_Fq.ec`, its invariants, and its termination proof.

Chapter 8 develops the algebraic infrastructure in `NTTFullAlgebra.ec`: primitive root properties, bit-reversal lemmas, and stage decompositions.

Chapter 9 describes the **Jasmin** implementation in detail.

Chapter 10 presents the Jasmin-to-EasyCrypt refinement proof.

Chapter 11 assembles all pieces into the end-to-end correctness theorems.

Chapter 12 discusses limitations, remaining obligations, related and future work.

Chapter 2

Background

2.1 Lattice-Based Cryptography

2.1.1 Module Lattices

A *lattice* $\Lambda \subseteq \mathbb{Z}^n$ is a discrete additive subgroup of $\mathbb{Z}^n$; equivalently, it is the $\mathbb{Z}$ -span of a basis matrix $\mathbf{B} \in \mathbb{Z}^{n \times n}$. The *shortest vector problem* (SVP) asks for the shortest non-zero lattice vector under the Euclidean norm, and is conjectured to be hard for quantum computers.

Module lattices generalise to modules over a polynomial ring. Let $R = \mathbb{Z}[X]/(X^n + 1)$ and $R_q = R/qR$. A *module lattice* of rank k over R is a free R -module $M \cong R^k$ together with a $\mathbb{Z}$ -embedding into $\mathbb{Z}^{kn}$. The *Module Short Integer Solution* (MSIS) and *Module Learning With Errors* (MLWE) problems over such lattices underpin the security of Dilithium, FALCON, and HAETAE.

2.1.2 NTT-Friendly Parameters

For efficient computation, the ring R_q must support a fast polynomial multiplication algorithm. If $q \equiv 1 \pmod{2n}$ then $X^n + 1$ splits completely over $\mathbb{F}_q$ and the NTT reduces polynomial multiplication to componentwise multiplication of length- n vectors. If only $q \equiv 1 \pmod{n}$ (but $q \equiv 3 \pmod{4}$ or similar), a *negacyclic* NTT can still be defined [LN16].

For HAETAE, $n = 256$ and $q = 64513 \equiv 1 \pmod{512}$. This means a primitive 512th

root of unity ζ_0 exists in $\mathbb{F}_q$, enabling a full 256-point negacyclic NTT.

2.2 Number-Theoretic Transform

2.2.1 Definition

Let q be an odd prime with $q \equiv 1 \pmod{2n}$, and let $\zeta \in \mathbb{F}_q^*$ be a primitive $2n$ th root of unity (so $\zeta^{2n} = 1$, $\zeta^n = -1$). The *negacyclic NTT* of a polynomial $f = \sum_{j=0}^{n-1} f_j X^j \in R_q$ is the vector $\hat{f} \in \mathbb{F}_q^n$ defined by

$$\hat{f}_i = \sum_{j=0}^{n-1} f_j \cdot \zeta^{(2\text{brev}_8(i)+1) \cdot j}, \quad i = 0, \dots, n-1, \quad (2.1)$$

where $\text{brev}_8 : \{0, \dots, 255\} \rightarrow \{0, \dots, 255\}$ is the bit-reversal permutation of an 8-bit index.

The inverse NTT is

$$f_i = \frac{1}{n} \sum_{j=0}^{n-1} \hat{f}_j \cdot \zeta^{-((2\text{brev}_8(j)+1) \cdot i)}, \quad i = 0, \dots, n-1. \quad (2.2)$$

Proposition 2.1 (Round-trip identity). $\text{INTT}(\text{NTT}(f)) = f$ for all $f \in R_q$.

2.2.2 Cooley–Tukey Butterfly Algorithm

The NTT defined by eq. (2.1) can be computed in $O(n \log n)$ field operations using the Cooley–Tukey decimation-in-time (DIT) algorithm. For the negacyclic case with $n = 256 = 2^8$, the algorithm proceeds in 8 stages.

At stage s (with $s = 0, 1, \dots, 7$, from outermost to innermost), the half-length is $\ell = 2^{7-s}$. If g is the group number within that stage, then the implementation’s table counter is $k = 2^s + g$ and the twiddle factor is

$$w = \zeta^{\text{brev}_8(k)}.$$

Index 0 of the concrete forward table is padding, so the loop consumes indices

$1, \dots, 255$. The butterfly operation on a pair (a, b) with twiddle factor w is:

$$(a', b') = (a + w \cdot b, a - w \cdot b). \quad (2.3)$$

Algorithm 1 Forward NTT (Cooley–Tukey DIT, negacyclic, $n = 256$)

```

1: procedure NTT( $a[0..255]$ )
2:    $k \leftarrow 1$ 
3:   for  $\ell = 128, 64, 32, 16, 8, 4, 2, 1$  do
4:     for  $start = 0, 2\ell, 4\ell, \dots, 254$  do
5:        $w \leftarrow \zeta^{\text{brev}_8(k)}$   $\triangleright k$ -th twiddle factor
6:        $k \leftarrow k + 1$ 
7:       for  $j = start, \dots, start + \ell - 1$  do
8:          $t \leftarrow w \cdot a[j + \ell] \bmod q$ 
9:          $a[j + \ell] \leftarrow a[j] - t$ 
10:         $a[j] \leftarrow a[j] + t$ 
11:      end for
12:    end for
13:  end for
14: end procedure

```

2.2.3 Negacyclic NTT vs. Standard NTT

A standard (cyclic) DFT of length n uses the recurrence for the cyclic ring $\mathbb{Z}_q[X]/(X^n - 1)$. The *negacyclic* ring $\mathbb{Z}_q[X]/(X^n + 1)$ requires a pre-twist by powers of ζ (a half-integer shift of the exponent), which accounts for the $+1$ in the exponent formula of eq. (2.1).

Critically for this dissertation, HAETAE uses a *complete* $n = 256$ negacyclic NTT with bit-reversal exponent $\text{brev}_8(k)$, whereas MLKEM uses an incomplete 128-point transform with exponent $\text{brev}_7(k)$. This distinction has direct consequences for which algebraic lemmas can be shared between verification projects.

2.3 Montgomery Modular Arithmetic

2.3.1 REDC Algorithm

Montgomery reduction [Mon85] is a technique for efficient modular multiplication that avoids expensive division. Given $a \in \mathbb{Z}$ with $0 \leq a < q \cdot R$ and integers q, R

with $\gcd(q, R) = 1$, the Montgomery product is $a \cdot R^{-1} \bmod q$.

Algorithm 2 Montgomery Reduction with the HAETAE Sign Convention

Require: $a, R = 2^{32}, q^+ = q^{-1} \bmod R$

Ensure: $a \cdot R^{-1} \bmod q$

- 1: $m \leftarrow (a \bmod R) \cdot q^+ \bmod R$
 - 2: $t \leftarrow (a - m \cdot q) \div R$
 - 3: **if** $t < 0$ **then return** $t + q$
 - 4: **else return** t
 - 5: **end if**
-

In the HAETAE implementation, $R = 2^{32}$, $q = 64513$, $\text{QINV} = 940508161$ ($\equiv q^{-1} \bmod 2^{32}$), and $\text{MONT} = R \bmod q = 14321$. Thus the implementation uses the positive-inverse convention in Algorithm 2: it computes $m = (a \bmod R)\text{QINV} \bmod R$ and then forms $a - mq$.

2.3.2 Montgomery Multiplication

Given $a, b \in \mathbb{Z}_q$ with both represented in Montgomery form (i.e., as $a \cdot R \bmod q$ and $b \cdot R \bmod q$), their product in Montgomery form is computed as $\text{REDC}(a \cdot b)$, since $\text{REDC}(aR \cdot bR) = abR^2 \cdot R^{-1} = abR \bmod q$.

2.3.3 Signed Montgomery Reduction

The HAETAE implementation uses a *signed* variant where intermediate values are 32-bit signed integers and the final conditional subtraction is replaced by an arithmetic right shift. Specifically, for $a \in \mathbb{Z}$ with $|a| < q \cdot 2^{15}$:

$$a \leftarrow \text{int32}(a_{10} \cdot \text{QINV}) \cdot q \quad (\text{as signed 64-bit}) \text{ shifted right by 32} \quad (2.4)$$

yields a value congruent to $a \cdot R^{-1} \bmod q$ and, in the checked signed REDC theorem, a representative in $[-q, q)$.

2.4 The EasyCrypt Proof Assistant

2.4.1 Architecture

EasyCrypt [BGB09] is an interactive proof assistant specialised for reasoning about cryptographic programs. It combines:

- A *program logic* layer supporting Hoare logic (HL) for deterministic programs and probabilistic relational Hoare logic (pRHL) for pairs of probabilistic programs.
- An *ambient logic* layer that is a form of higher-order logic (HOL) with a type theory supporting algebraic structures (groups, rings, fields).
- A tactic engine with automation backends including SMT solvers (CVC4, Z3) and a computer algebra system for ring/field equalities.

2.4.2 Hoare Logic in EasyCrypt

A *Hoare triple* $\{P\} c \{Q\}$ asserts that if precondition P holds before executing command c , then postcondition Q holds after. In EasyCrypt, Hoare triples over *procedures* (modules with state) are written `hoare[M.f : P ==> Q]`.

A *relational Hoare triple* $\{P\} (c_1 \sim c_2) \{Q\}$ asserts that if c_1 and c_2 start in states satisfying P , their outputs satisfy Q . This is the foundation for proving that two programs are equivalent.

2.4.3 Module System

EasyCrypt organises code into *modules*. A module interface (type) is a signature listing procedure names and types; a module is a concrete implementation. The Jasmin compiler generates an EasyCrypt module for each exported function, which can then be related to hand-written specification modules via pRHL.

2.4.4 Theory Cloning

Algebraic structures are developed as parameterised *theories* that are instantiated by *cloning*: `clone Ring.ComRing as Zq with type t = int, ...` creates a copy of the commutative ring theory with $\mathbb{Z}_q$ as the carrier type and appropriate witnesses for the ring axioms.

2.5 The Jasmin Programming Language

Jasmin [Alm+17] is a C-like language designed for high-assurance cryptographic code. Its distinguishing features are:

- **Explicit register allocation.** Variables are annotated `reg` (in a register), `stack` (on the stack), or `ptr` (a pointer). The compiler enforces the annotation.
- **Data types matching machine types.** The types `u32`, `u64`, `u256` correspond directly to 32-bit, 64-bit, and 256-bit (AVX2) machine words.
- **Safety verification.** A safety checker ensures no undefined behaviour (array out of bounds, use before initialisation, integer overflow in unsigned arithmetic).
- **Constant-time verification.** The companion `jasmin-ct` checker verifies that secret values do not control branches or memory addresses. Its speculative mode also tracks transient pointer inputs, using explicit `#[ct = "..."]` and `#[sct = "..."]` contracts on exported procedures.
- **EasyCrypt extraction.** Each `export fn` is compiled to both assembly and an EasyCrypt module. The extracted module has a semantic identical to the assembly, by the Jasmin compiler correctness theorem.

```
1 export fn poly_ntt_jazz(reg mut ptr u32[256] rp)
2   -> reg ptr u32[256]
3 {
4   rp = _poly_ntt(rp);
5   return rp;
6 }
```

Listing 2.1: Jasmin function signature example

The export annotation triggers both assembly generation and `EasyCrypt` extraction. The extracted `EasyCrypt` type for `ptr u32[256]` is a `BArray1024.t` byte-array value, with coefficients accessed by `BArray1024.get32` and `BArray1024.set32`.

2.6 Related Work

MLKEM/Kyber NTT. The closest predecessor is the Jasmin/`EasyCrypt` verification of the NTT in ML-KEM [Bar+21a]. That work verifies the incomplete 128-point Cooley–Tukey NTT over a related ring structure but with an incomplete transform and a different twiddle-indexing discipline. Our work adapts techniques from that development but requires new algebra lemmas for the full 256-point transform.

Dilithium NTT. The `EasyCrypt` verification of Dilithium [Bar+21b] also formalises an NTT in $\mathbb{Z}_q[X]/(X^{256} + 1)$ but uses a different prime $q = 8380417$ with different twiddle factor structure.

FALCON. FALCON uses a floating-point NTT over $\mathbb{Z}[X]/(X^n + 1)$ [Fou+18]. Its Jasmin implementation has been verified using `EasyCrypt` but the field arithmetic is quite different from the integer case studied here.

High-assurance cryptography. The HACL* project [Zin+17] produces C code verified in F^*/Low^* , with machine-checked compilation to assembly. While HACL* has verified ChaCha20 and Poly1305, it has not (at time of writing) produced a verified lattice-based NTT. Our work contributes to the complementary Jasmin/`EasyCrypt` ecosystem, which has better support for the bitwise operations prevalent in lattice primitives.

Chapter 3

The HAETAE Signature Scheme and Its NTT

3.1 Overview of HAETAE

HAETAE [HAE22] is a module-lattice based digital signature scheme submitted to the Korean Post-Quantum Cryptography (KPQC) standardisation process. It follows the “Fiat–Shamir with aborts” paradigm [Lyu12]: the signing algorithm samples a masking polynomial, computes a commitment via a hash function, and rejects the sample if the output leaks information about the secret key.

3.1.1 Algebraic Setting

HAETAE operates in the polynomial ring

$$R_q = \mathbb{Z}_q[X]/(X^n + 1), \quad n = 256, \quad q = 64513. \quad (3.1)$$

Keys and signatures are vectors in R_q^k for parameter-set-dependent ranks k . Polynomial multiplication in R_q is the dominant operation in key generation, signing, and verification.

3.1.2 Role of the NTT

The NTT converts polynomials from the *coefficient domain* to the *evaluation domain*, where multiplication is pointwise. The workflow is:

$$f \cdot g \in R_q \longleftrightarrow \text{NTT}(f) \odot \text{NTT}(g) \in \mathbb{F}_q^n, \quad (3.2)$$

where $\odot$ denotes componentwise multiplication. A single polynomial multiplication thus costs two forward NTTs, one componentwise multiplication, and one inverse NTT, for overall $O(n \log n)$ field operations instead of $O(n^2)$.

3.2 HAETAE NTT Parameters

Table 3.1: HAETAE NTT parameters

Symbol	Value	Description
q	64513	Modulus ($= 2^8 \cdot 252 + 1 = 252 \cdot 256 + 1$)
n	256	Polynomial degree
ζ_0	426	Primitive 512th root of unity in $\mathbb{F}_q$
R	2^{32}	Montgomery base
QINV	940508161	$q^{-1} \bmod 2^{32}$
MONT	14321	$R \bmod q = 2^{32} \bmod q$
MONTSQ	4214	$R^2 \bmod q = 2^{64} \bmod q$
R^{-1}	50386	$R^{-1} \bmod q$
f	-29720	Inverse NTT scaling constant (in Montgomery form)

3.2.1 Twiddle Factors

The forward NTT uses 256 table slots in the reference array `zetas[256]`. Slot 0 is a padding value and the forward loop reads indices $1, \dots, 255$. For $1 \leq k < 256$, the concrete table entry is the signed 32-bit representative of $\text{MONT} \cdot \zeta_0^{\text{brev}_8(k)} \bmod q$.

The C inverse NTT reads the forward table in reverse order as `-zetas[-k]`. The Jasmin extraction stores the same inverse butterfly constants explicitly as `jzetas_inv[0..254]` and reserves `jzetas_inv[255]` for the final scaling constant $f = n^{-1} \cdot \text{MONT}^2 \bmod q = -29720$.

Table 3.2: First sixteen forward twiddle factors `zetas[1..16]` (signed Montgomery form)

k	1	2	3	4	5	6	7	8
<code>zetas[k]</code>	26964	-16505	22229	30746	20243	19064	-31218	9395
k	9	10	11	12	13	14	15	16
<code>zetas[k]</code>	-30985	22859	-8851	32144	13744	21408	17599	-16039

3.3 Reference C Implementation

The reference C implementation in `ntt.c` provides the algorithmic ground truth. It implements the forward NTT (Algorithm 1) and the inverse NTT with an extra final scaling step.

3.3.1 Forward NTT

```

1 void ntt(int32_t a[256]) {
2     unsigned int len, start, j, k;
3     int32_t t, zeta;
4
5     k = 1;
6     for (len = 128; len > 0; len >>= 1) {
7         for (start = 0; start < 256; start += 2 * len) {
8             zeta = zetas[k++];
9             for (j = start; j < start + len; j++) {
10                t = montgomery_reduce((int64_t)zeta * a[j + len]);
11                a[j + len] = a[j] - t;
12                a[j] = a[j] + t;
13            }
14        }
15    }
16 }
```

Listing 3.1: Forward NTT from `ntt.c`

The loop nest has three levels:

1. **Outer loop:** $\ell \in \{128, 64, 32, 16, 8, 4, 2, 1\}$, the current butterfly half-length. Each iteration is one NTT stage.

2. **Middle loop:** iterates over the $256/(2\ell)$ groups within each stage.
3. **Inner loop:** performs ℓ butterfly operations within each group, using the same twiddle factor `zeta = zetas[k]`.

Key observations:

- After stage s (with $\ell = 2^{7-s}$), the array is arranged so that each processed segment represents the checked `partial_ntt` sum for the current segment length. The formal invariant is indexed by $(\ell, start, bsj)$, not by a congruence class modulo 2^ℓ .
- The twiddle counter k advances monotonically from 1 to 255, consuming all 255 non-trivial twiddle factors.
- No modular reduction is applied at the output of butterfly pairs. The checked proof tracks stage-dependent signed bit-width bounds: the forward refinement theorem assumes a 16-bit input bound and proves a 24-bit output bound.

3.3.2 Inverse NTT

```

1 void invntt_tomont(int32_t a[256]) {
2     unsigned int start, len, j, k;
3     int32_t t, zeta;
4     const int32_t f = -29720;    /* mont^2 / 256 mod q */
5
6     k = 256;
7     for (len = 1; len < 256; len <= 1) {
8         for (start = 0; start < 256; start = j + len) {
9             zeta = -zetas[--k];
10            for (j = start; j < start + len; j++) {
11                t = a[j];
12                a[j] = t + a[j + len];
13                a[j+len] = t - a[j+len];
14                a[j+len] = montgomery_reduce((int64_t)zeta * a[j+
15                    len]);
16            }
17        }
18    }
19    for (j = 0; j < 256; j++)

```

```

19     a[j] = montgomery_reduce((int64_t)f * a[j]);
20 }

```

Listing 3.2: Inverse NTT from `ntt.c`

The inverse NTT reverses the butterfly ordering: ℓ increases from 1 to 128, and the C twiddle counter starts at 256 so that the first butterfly uses `-zetas[255]`. The final loop multiplies each coefficient by $f = -29720 \equiv n^{-1} \cdot R^2 \pmod{q}$, with the Montgomery reduction removing one factor of R and leaving each coefficient multiplied by $n^{-1} \cdot R \pmod{q}$ (i.e., in Montgomery form).

3.3.3 Montgomery Reduction

```

1 int32_t montgomery_reduce(int64_t a) {
2     int32_t t;
3     t = (int32_t)a * (int32_t)QINV;
4     t = (a - (int64_t)t * Q) >> 32;
5     return t;
6 }

```

Listing 3.3: Montgomery reduction from `reduce.c`

This implements the signed Montgomery reduction. The cast `(int32_t)a` extracts the low 32 bits of a (interpreted as a signed 32-bit integer), the product `t * Q` is computed as a signed 64-bit value, and the right shift by 32 is an arithmetic right shift on signed integers.

3.3.4 Test Harness

The file `test_haetae_ntt.c` provides a reference test harness that:

1. Generates random polynomial inputs within the valid range.
2. Compares the C forward NTT with the **Jasmin** forward NTT.
3. Compares the C inverse NTT with the **Jasmin** inverse NTT.
4. Checks the combined C and **Jasmin** forward–inverse pipelines agree.

These tests provide empirical C/Jasmin agreement. The round-trip claim used for interpretation is the mathematical abstract inverse identity together with the two separate EasyCrypt end-to-end theorems when the inverse theorem’s input-bound precondition is satisfied; it is not inferred from this test harness alone, and it is not packaged as a standalone checked pipeline theorem.

3.4 Structural Comparison with MLKEM NTT

Table 3.3: Comparison of HAETAE and MLKEM NTT structures

Property	HAETAE	MLKEM
Modulus q	64513	3329
Degree n	256	256
NTT type	Full 256-point negacyclic	Incomplete 128-point
Bit-reversal exponent	$\text{brev}_8(k)$	$\text{brev}_7(k)$
Primitive root	$\zeta_0 = 426$ (512th)	$\zeta_0 = 17$ (256th)
Output permutation	Bit-reversed	Bit-reversed
Final inv. NTT step	Scale by $n^{-1}R \bmod q$	Scale by $n^{-1}R \bmod q$

The key structural difference is that the HAETAE NTT uses twiddle factors $\zeta^{\text{brev}_8(k)}$ where k ranges from 1 to 255, producing a full 256-point spectral evaluation. MLKEM uses twiddle factors $\zeta^{\text{brev}_7(2k+1)}$, producing only a 128-point evaluation (the polynomial is split into even/odd parts). This means the HAETAE NTT result vector contains evaluation at all 256 primitive 512th roots of unity, while the MLKEM result vector contains evaluations at 128 primitive 256th roots.

This structural difference invalidates any attempt to directly reuse MLKEM NTT algebra lemmas for the HAETAE verification. New algebraic infrastructure is required, developed in `NTTFullAlgebra.ec` (Chapter 8).

Chapter 4

Formal Verification Framework

4.1 Overview of the Verification Architecture

The verification of the HAETAET NTT is structured as a *vertical refinement chain*: a sequence of proof steps, each relating an implementation or specification at one abstraction level to a specification at the next. Figure 4.1 illustrates the overall structure.

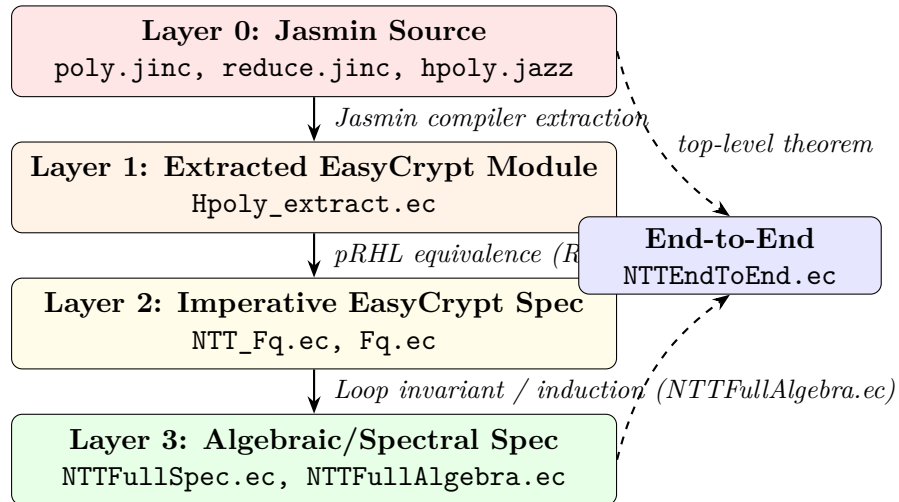


Figure 4.1: Four-layer verification architecture for the HAETAET NTT. Solid arrows are machine-checked proof steps; dashed arrows indicate the end-to-end composition.

4.2 Layer 0: Jasmin Source

The Jasmin source files implement the NTT algorithm in a style close to assembly but with structured control flow. The key files are:

- `poly.jinc`: The main NTT and inverse-NTT kernels, implemented as `inline fn _poly_ntt` and `inline fn _poly_invntt`.
- `reduce.jinc`: The Montgomery reduction primitive `__montgomery_reduce` and the derived `__fqmul`.
- `zetas.jinc`: The 256-element twiddle-factor table as a global constant.
- `hpoly.jazz`: The exported entry points `export fn poly_ntt_jazz` and `export fn poly_invntt_jazz`.

The Jasmin compiler verifies safety properties (no array-out-of-bounds, no use-before-initialisation) and then simultaneously generates:

1. x86-64 assembly (`hpoly.s`).
2. An EasyCrypt module `Hpoly_extract.ec` that is semantically equivalent to the assembly by the Jasmin compiler correctness theorem [Alm+17].

4.3 Layer 1: Extracted EasyCrypt Module

The extracted module `Hpoly_extract.ec` (`extract/` folder) contains EasyCrypt procedure definitions that mirror the Jasmin structure. A fragment is shown in Listing 4.1.

```
1 module M = {  
2   proc poly_ntt_jazz (rp : BArray1024.t) :  
3     BArray1024.t = {  
4     var zeta : W32.t;  
5     var t     : W32.t;  
6     var len   : int;  
7     var start: int;  
8     var j     : int;  
9     (* ... body mirrors Jasmin structure ... *)
```

```

10   return rp;
11   }
12 }.

```

Listing 4.1: Fragment of extracted procedure in Hpoly_extract.ec

This module is the “ground truth” for what the compiled assembly computes. All subsequent proof obligations are stated with respect to this module.

4.4 Layer 2: Imperative EasyCrypt Specification

The file `NTT_Fq.ec` contains hand-written EasyCrypt procedures that closely mirror the reference C implementation. Unlike the extracted module, these procedures work with abstract coefficient arrays over $\mathbb{F}_q$ (type `coeff Array256.t`) rather than machine-word arrays.

The translation from machine words to field coefficients is captured by the *word-to-coefficient* relation `poly_repr`: a concrete `BArray1024.t` word array corresponds to a coefficient array when each loaded 32-bit word maps to the same $\mathbb{F}_q$ element. The strengthened `poly_repr_bound` predicate also records the signed bit-width bound required for word arithmetic.

The imperative specification exposes the full loop structure, making it amenable to Hoare-style invariant reasoning.

4.5 Layer 3: Abstract Spectral Specification

The files `NTTFullSpec.ec` and `NTTFullAlgebra.ec` define the NTT at the level of abstract algebra. No loop structure is visible; the NTT is defined as a linear evaluation map over $\mathbb{F}_q^{256}$.

Definition 4.1 (Abstract forward NTT, `NTTFullSpec.full_ntt`). Let $p : \mathbb{F}_q^{256}$. The forward NTT of p is the map

$$\text{NTT}(p)[i] = \sum_{j=0}^{255} p[j] \cdot \zeta_0^{(2\text{brevs}(i)+1) \cdot j} \pmod{q}, \quad i = 0, \dots, 255.$$

Definition 4.2 (Abstract inverse NTT, `NTTFullSpec.full_invntt`). Let $p : \mathbb{F}_q^{256}$. The inverse NTT of p is the map

$$\text{INTT}(p)[i] = \frac{1}{256} \sum_{j=0}^{255} p[j] \cdot \zeta_0^{-((2\text{brevs}(j)+1) \cdot i)} \pmod{q}, \quad i = 0, \dots, 255.$$

4.6 End-to-End Composition

The file `NTTEndToEnd.ec` imports all three layers and states the main correctness theorems as Hoare judgements for the exported `Jasmin` procedures:

```

1 lemma haetae_jasmin_ntt_correct p :
2   hoare [Hpoly_extract.M.poly_ntt_jazz :
3     NTT_Fq.poly_repr_bound rp p 16 ==>
4     NTT_Fq.poly_repr_bound res (NTTFullSpec.full_ntt p)
5     24] .
6
7 lemma haetae_jasmin_invntt_correct p :
8   hoare [Hpoly_extract.M.poly_invntt_jazz :
9     NTT_Fq.poly_repr_bound rp p 16 ==>
10    NTT_Fq.poly_repr_bound res
11      (NTT_Fq.array256_mont (NTTFullSpec.full_invntt p))
12      16] .

```

Listing 4.2: End-to-end theorem statement in `NTTEndToEnd.ec`

These theorem statements capture the vertical refinement: starting from a `Jasmin` export whose input array satisfies `poly_repr_bound`, the forward output represents the abstract spectral NTT with a 24-bit bound, and the inverse output represents the abstract inverse NTT in Montgomery form with a 16-bit bound.

4.7 Proof Methodology

4.7.1 Proceeding Bottom-Up

We build proofs bottom-up, starting with the most concrete layer and moving toward the most abstract. This order is forced by the dependency structure: the pRHL refinement proof (Layer 1 $\rightarrow$ Layer 2) requires that the imperative specification (Layer 2) is already proved correct with respect to the abstract specification (Layer 2 $\rightarrow$ Layer 3).

4.7.2 Hoare Logic for Loop Invariants

The main technical tool for Layers 2–3 is *loop invariant reasoning*. Each NTT stage is characterised by an invariant asserting that a partial set of coefficients has been transformed. The invariant is a conjunction of:

1. A *partial NTT* condition: indices below the current counter have been transformed through the current stage.
2. A *bounds condition*: transformed coefficients remain within the established bit-width bounds.
3. A *twiddle-factor synchronisation* condition: the twiddle counter matches the current loop indices.

4.7.3 pRHL for Procedure Equivalence

The refinement from the extracted module to the imperative specification uses pRHL judgements of the form

$$\begin{aligned} & \text{Pr}[\text{Hpoly_extract.M. poly_ntt}(\text{rp}) \rightarrow \text{rp}'] \\ &= \text{Pr}[\text{NTT_Fq.NTT.ntt}(\text{a}, \text{zetas}) \rightarrow \text{a}'], \quad \text{given } \text{poly_repr_bound}(\text{rp}, \text{a}, 16). \end{aligned}$$

where `poly_repr_bound` relates the concrete word array `rp` to the abstract field array `a` and records the signed bit-width bound. Since both procedures are deterministic, this reduces to a functional equivalence assertion.

4.7.4 Field-Theory Automation

The abstract-level proofs rely on **EasyCrypt**'s theory-cloning mechanism to instantiate the standard field axioms for $\mathbb{F}_q$ and to invoke automated tactics for polynomial identities. Long algebraic chains are dispatched by the **field** and **ring** tactics, backed by CVC5 and Z3 for non-linear arithmetic goals.

4.8 File Dependency Graph

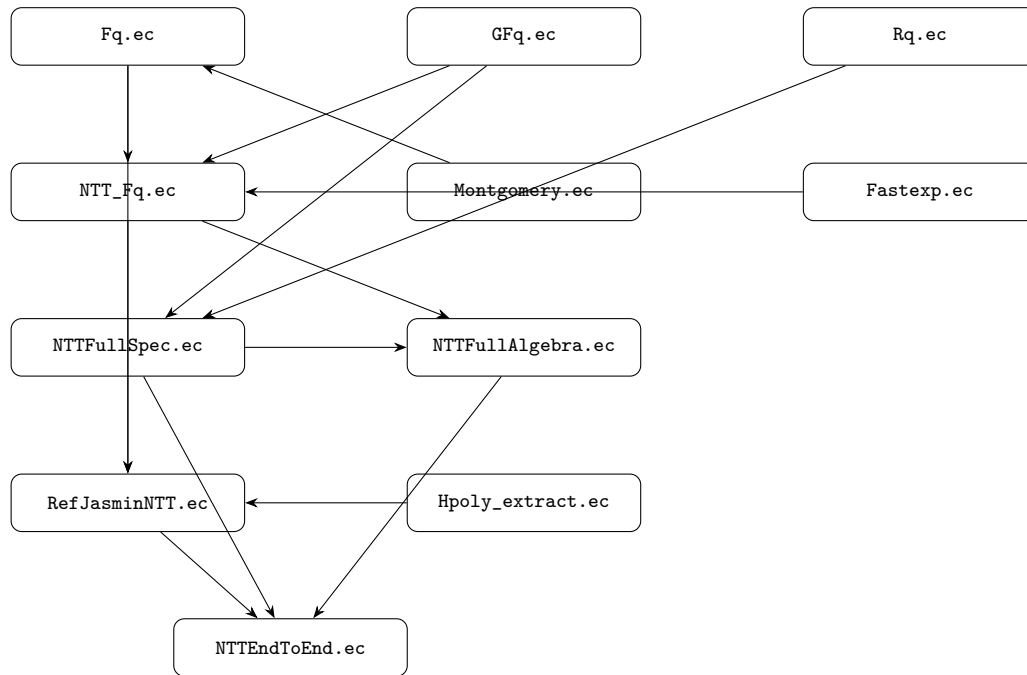


Figure 4.2: Core EasyCrypt file dependency graph for the HAETAET NTT verification. Arrows point from dependency to dependent; low-level Jasmin array clones and standard-library theories are omitted here and recorded in Table 4.1.

4.9 Relationships Between the EasyCrypt Files

Figure 4.2 gives the main proof-chain dependencies. The table below records the role of each project `.ec` file and the reason it appears in the checked development. This distinction matters because not every import has the same proof meaning: some files provide representation infrastructure, some state executable procedures, and the last files compose already-checked judgements into the final theorem.

Table 4.1: Relationships between the EasyCrypt source files

File	Layer / role	Relationship to the proof
Array256.ec	Array infrastructure	Clones the Jasmin fixed-size array theory at size 256. It is the common container for polynomials, twiddle tables, and abstract NTT outputs.
BArray1024.ec and WArray1024.ec	Concrete memory infrastructure	Instantiate byte-array and word-array theories used by the extracted Jasmin module. They support table reads, word loads, and stores in <code>Hpoly_extract.ec</code> and <code>RefJasminNTT.ec</code> .
GFq.ec	Field foundation	Defines the HAETAE modulus $q = 64513$ and clones the finite-field theory. It supplies the coefficient type used by <code>Rq.ec</code> , <code>NTT_Fq.ec</code> , <code>NTTFullSpec.ec</code> , and <code>NTTFullAlgebra.ec</code> .
Rq.ec	Polynomial-ring vocabulary	Defines the polynomial type, ring operations, root ζ , bit-reversal notation, and big-sum infrastructure. Its older <code>ntt</code> operator is kept for comparison, while the final target is the full transform in <code>NTTFullSpec.ec</code> .
Montgomery.ec	Arithmetic foundation	Contains modular-reduction lemmas and Montgomery arithmetic facts used to prove the word-level routines in <code>Fq.ec</code> and reused by the Jasmin refinement proofs.

File	Layer / role	Relationship to the proof
Fq.ec	Word arithmetic correctness	Proves correctness of the extracted Montgomery reduction, field multiplication, addition, subtraction, and signed-word bounds. These lemmas discharge the arithmetic side conditions inside RefJasminNTT.ec.
Fastexp.ec	Exponentiation support	Provides generic finite-field exponentiation lemmas. NTT_Fq.ec clones this theory for the HAETAE coefficient field.
Hpoly_extract.ec	Extracted Jasmin model	Contains the EasyCrypt procedures generated from the Jasmin implementation: poly_ntt_jazz, poly_invntt_jazz, their internal cores, and the concrete twiddle tables. It is the concrete side of the refinement proof.
NTT_Fq.ec	Imperative specification	Defines the hand-written coefficient-level procedures NTT.ntt and NTT.invntt, the mathematical twiddle tables, and the representation predicates such as poly_repr_bound. It is the target of RefJasminNTT.ec and the source of NTTFullAlgebra.ec.
NTTFullSpec.ec	Closed-form abstract specification	Defines full_ntt and full_invntt as the full 256-point HAETAE transforms. It also records that Rq.ntt is not this full transform, preventing the end-to-end theorem from targeting the wrong operator.

File	Layer / role	Relationship to the proof
NTTFullAlgebra.ec	Imperative-to-abstract bridge	Proves the Hoare statements that running <code>NTT_Fq.NTT.ntt</code> with the checked twiddle table returns <code>NTTFullSpec.full_ntt</code> , and similarly for the inverse transform. This file closes the algebraic loop-invariant part of the proof.
RefJasminNTT.ec	Jasmin-to-imperative refinement	Proves pRHL equivalences between the extracted Jasmin cores in <code>Hpoly_extract.ec</code> and the imperative procedures in <code>NTT_Fq.ec</code> . It also lifts the internal-core result to the public Jasmin wrappers.
NTTEndToEnd.ec	Final composition	Imports <code>RefJasminNTT.ec</code> , <code>NTTFullAlgebra.ec</code> , <code>NTTFullSpec.ec</code> , <code>NTT_Fq.ec</code> , and <code>Hpoly_extract.ec</code> . It composes the refinement and algebraic Hoare lemmas into the final forward and inverse Jasmin NTT correctness statements.

In short, the proof has two independent bridges that meet in `NTTEndToEnd.ec`. The concrete bridge is

$$\text{Hpoly_extract.ec} \xrightarrow{\text{RefJasminNTT.ec}} \text{NTT_Fq.ec},$$

which says that the extracted Jasmin procedures implement the imperative coefficient-level model. The algebraic bridge is

$$\text{NTT_Fq.ec} \xrightarrow{\text{NTTFullAlgebra.ec}} \text{NTTFullSpec.ec},$$

which says that the imperative model computes the full closed-form HAETAE NTT. The end-to-end file imports both bridges and composes them.

Chapter 5

Formalisation of Field and Ring Arithmetic

5.1 The Prime Field $\mathbb{F}_q$

5.1.1 Field Definition in `Fq.ec`

The file `Fq.ec` (535 lines) provides the signed-reduction and Montgomery-arithmetic infrastructure for the modulus $q = 64513$. The field structure itself is imported through the neighbouring `GFq.ec` file; `Fq.ec` clones the `SignedReductions` theory with the concrete HAETAE constants so that word-level signed arithmetic can be related to field coefficients.

```
1  require import AllCore List IntDiv CoreMap.
2  from Jasmin require import JWord JModel.
3  require import Montgomery.
4  require import Hpoly_extract.
5
6  op q : int = 64513.
7  lemma qE : q = 64513 by rewrite /q.
8
9  clone import SignedReductions with
10     op k      <- 32,
11     op q      <- q,
12     op qinv   <- 940508161,
13     op Rinv   <- 50386
```

```

14   proof q_bnd      by (rewrite /R qE => />)
15   proof q_odd1     by (rewrite qE => />)
16   proof q_odd2     by (rewrite qE => />)
17   proof qqinv      by (rewrite /R qE => />)
18   proof Rinv_gt0   by (auto => />)
19   proof RRinv      by (rewrite /R qE => />)
20   proof qinv_bnd   by (rewrite /R => />).

```

Listing 5.1: Field declaration in Fq.ec

5.1.2 Signed vs. Unsigned Representation

A key design decision in the refinement proof is to separate field equality from machine-word signed bounds. Modular residues such as $a \bmod q$ are not treated as centered representatives. Instead, concrete u32 coefficients are mapped to field elements by `word_to_coeff`, while side conditions such as `Fq.bw32 w sz` record the signed interpretation $-2^{sz} \leq \text{W32.to_sint } w < 2^{sz}$.

The file defines the *signed word conversion* operator:

$$\text{to_sint}(w) = \begin{cases} w & \text{if } w < 2^{31} \\ w - 2^{32} & \text{otherwise} \end{cases}$$

which maps a 32-bit unsigned word to its signed interpretation, and the inverse `of_sint` which lifts a bounded integer to a 32-bit word.

5.1.3 Bit-Width Bounds

A central technical device in Fq.ec is the *bit-width bound predicate*:

$$\text{bw32}(w, s) \Leftrightarrow |\text{to_sint}(w)| < 2^s,$$

for a bit-width parameter $s \in \{1, \dots, 31\}$. This captures the invariant that a machine word represents a “small” integer.

Lemma 5.1 (Bound addition). *If $\text{bw32}(a, s_a)$ and $\text{bw32}(b, s_b)$, then $\text{bw32}(a + b, \max(s_a, s_b) + 1)$.*

Lemma 5.2 (Bound multiplication). *If $\text{bw32}(a, s_a)$ and $\text{bw32}(b, s_b)$, then $\text{bw64}(a \cdot b, s_a + s_b)$ where `bw64` is the 64-bit analogue.*

These lemmas directly support the analysis of intermediate products in the NTT butterfly: a 16-bit coefficient multiplied by a 15-bit twiddle factor gives a 31-bit product, safe in a 32-bit signed word.

5.1.4 Signed Arithmetic Lemmas

The EasyCrypt standard library does not include arithmetic lemmas for signed integer interpretations of machine words. Fq.ec contributes several new lemmas, including:

Lemma 5.3 (Signed right shift). *For $a \in \mathbb{Z}$ with $|a| < 2^{63}$,*

$$\text{to_sint}(a \gg_s 32) = a \div 2^{32},$$

where $\gg_s$ denotes arithmetic right shift and $\div$ is EasyCrypt's integer division operation, matching the floor-style semantics of an arithmetic shift on negative two's-complement values.

This lemma is crucial for formalising the Montgomery reduction formula, which uses an arithmetic right shift to divide by $R = 2^{32}$.

5.2 The $\mathbb{Z}_q$ Field via Theory Cloning

5.2.1 GFq.ec: Abstract Field

The file GFq.ec clones EasyCrypt's abstract ZModField theory with $q = 64513$:

```

1  require import AllCore.
2  require import IntDiv.
3  require import ZModP Ring.
4
5  op q : int = 64513 axiomatized by qE.
6  axiom prime_q : prime q.
7
8  clone import ZModField as Zq with
9    op p <- q
10   rename "zmod" as "coeff"
11   rename "ZModp" as "Zq"
12   proof prime_p by apply prime_q

```

```

13   proof *.
14
15   (* Signed representative *)
16   op as_sint(x : coeff) =
17     if (q-1) %/ 2 < asint x then asint x - q else asint x.

```

Listing 5.2: GFq.ec: Galois field instantiation

This instantiation provides the complete algebraic structure of $\mathbb{F}_q$: commutativity, associativity, distributivity, existence of inverses, and the characteristic- q axiom $q \cdot 1 = 0$.

5.2.2 Primitive Root

The primitive 512th root of unity $\zeta_0 = 426$ is defined and verified:

```

1   op zroot : Zq.zmod = inZq 426.
2
3   lemma zroot_512 : zroot ^ 512 = Zq.one.
4   lemma zroot_256 : zroot ^ 256 = Zq.(-\!one).
5   lemma zroot_primitive : forall k, 0 < k < 512 =>
6     zroot ^ k <> Zq.one.

```

Listing 5.3: Primitive root declaration in NTTFullAlgebra.ec

These are proved by direct computation in $\mathbb{F}_{64513}$, using the SMT backend to discharge the modular arithmetic obligations.

5.3 The Polynomial Ring R_q

5.3.1 Rq.ec: Ring Definition

The file Rq.ec (118 lines) defines the polynomial ring $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ as a 256-element coefficient vector over $\mathbb{Z}_q$:

```

1   require import Array256.
2
3   (* Polynomial = coefficient vector *)
4   type poly = Zq.zmod Array256.t.

```

```

5
6  (* Ring addition *)
7  op (&+) (p q : poly) : poly =
8      Array256.map2 Zq.(+) p q.
9
10 (* Ring multiplication (schoolbook, negacyclic) *)
11 op (&*) (p q : poly) : poly =
12     let r = Array256.create Zq.zero in
13     let r = foldr (fun i r =>
14         let pi = p.[i] in
15         foldr (fun j r =>
16             let pij = pi * q.[j] in
17             let idx = (i + j) %% 512 in
18             if idx < 256 then r.[idx] <- r.[idx] + pij
19                 else r.[idx - 256] <- r.[idx - 256] -
20                     pij]
21         ) r (iota_ 0 256)
22     ) r (iota_ 0 256) in
23     r.

```

Listing 5.4: Ring definition in Rq.ec

5.3.2 Legacy NTT Operators in Rq.ec

Rq.ec also contains legacy axiomatized operators `ntt` and `invntt`. They are kept as part of the surrounding ring interface, but they are not the full HAETAE NTT proved in this dissertation. The checked full-transform development uses the explicit spectral maps in `NTTFullSpec.ec` and proves the Jasmin NTT against those maps through `NTTFullAlgebra.ec`.

```

1  op ntt (p: poly) = ... axiomatized by nttE.
2  op invntt (p: poly) = ... axiomatized by invnttE.

```

Listing 5.5: Legacy NTT equations in Rq.ec

The distinction matters: `NTTFullSpec.ec` proves that this legacy `Rq.ntt` is not the same as the full HAETAE transform on a simple test polynomial. The end-to-end theorem in `NTTEndToEnd.ec` therefore uses `NTTFullSpec.full_ntt` and `NTTFullSpec.full_invntt`, not the legacy `Rq.ntt` operators.

5.4 Array Infrastructure

5.4.1 Array256.ec

The 256-element array type `Array256.t` is a typed abstract type with the following operations:

- `. [i]` and `. [i <- v]`: functional get and set.
- `map`, `map2`: pointwise operations.
- `iteri`: iteration with index.
- `init`: create from a function.
- `create`: create with constant value.

Key properties include extensional equality (`Array256.ext_eq`): two arrays are equal if and only if they agree at every index.

5.4.2 Word Array Conversion

The bridge between machine-word arrays and field-coefficient arrays is captured by the following predicates:

Definition 5.4 (Word-to-coefficient correspondence). Given a Jasmin memory array `rp : BArray1024.t` and a field array `p :  $\mathbb{F}_q^{256}$` , `poly_repr rp p` states:

$$\forall i \in [0, 256). \text{word_to_coeff}(\text{BArray1024.get32}(\text{rp}, i)) = p[i].$$

The strengthened predicate `poly_repr_bound rp p sz` adds a signed bit-width bound `Fq.bw32` for every loaded word.

This relation is the key linking the extracted `Jasmin` module (which works with `BArray1024.t` memory) to the imperative specification (which works with $\mathbb{F}_q$ coefficient arrays). It is the core of the refinement proof in `RefJasminNTT.ec`.

5.5 Reduction and Auxiliary Operations

5.5.1 Conditional Addition (**caddq**)

The **caddq** operation adds q to negative coefficients:

$$\text{caddq}(a) = \begin{cases} a + q & \text{if } a < 0 \\ a & \text{otherwise} \end{cases}$$

It maps signed representatives in $(-q, 0) \cup [0, q)$ to non-negative representatives in $[0, q)$, without modular reduction.

Lemma 5.5 (*caddq correctness*). *For all $a \in \mathbb{Z}$ with $-q < a < q$: $\text{caddq}(a) \equiv a \pmod{q}$ and $0 \leq \text{caddq}(a) < q$.*

5.5.2 Freeze (**freeze**)

Full modular reduction to $[0, q)$ is defined as:

$$\text{freeze}(a) = ((a \bmod q) + q) \bmod q.$$

Lemma 5.6 (*freeze correctness*). *For all $a \in \mathbb{Z}$: $\text{freeze}(a) \equiv a \pmod{q}$ and $0 \leq \text{freeze}(a) < q$.*

Chapter 6

Formalisation of Montgomery Arithmetic

6.1 Motivation and Design Choices

Montgomery reduction [Mon85] is the arithmetic backbone of the HAETAE NTT. Every butterfly operation calls `montgomery_reduce` to bring an intermediate 64-bit product back to a 32-bit representative modulo q . An incorrect implementation of this operation would silently corrupt all NTT outputs while passing many unit tests.

Two aspects make the formal verification non-trivial:

1. The reduction uses *signed* 32-bit integers in an arithmetic right shift, whose behaviour on negative values is implementation-defined in C but well-specified in Jasmin.
2. The output is not fully reduced (it lies in approximately $(-q, q)$ rather than $[0, q)$), and subsequent operations must account for this.

6.2 Montgomery Parameters

The Montgomery parameters for HAETAE are as follows. Let $R = 2^{32}$, $q = 64513$. Since $\gcd(q, R) = 1$, the inverse $R^{-1} \bmod q$ and $q^{-1} \bmod R$ both exist.

$$QINV = q^{-1} \bmod R = 940508161 \quad (6.1)$$

$$MONT = R \bmod q = 14321 \quad (6.2)$$

$$MONTSQ = R^2 \bmod q = 4214 \quad (6.3)$$

$$R^{-1} \bmod q = 50386 \quad (6.4)$$

Lemma 6.1 (Parameter verification). *The following hold in $\mathbb{Z}$:*

1. $q \cdot QINV \equiv 1 \pmod{2^{32}}$
2. $MONT \equiv R \pmod{q}$
3. $MONTSQ \equiv R^2 \pmod{q}$
4. $R^{-1} \cdot R \equiv 1 \pmod{q}$

Proof. All four identities are discharged by concrete computation in EasyCrypt using the `smt` tactic with Z3. □

6.3 Signed Montgomery Reduction

6.3.1 Algorithm

The HAETAE Montgomery reduction operates on a signed 64-bit integer a :

```

1 int32_t montgomery_reduce(int64_t a) {
2     int32_t t;
3     t = (int32_t)a * (int32_t)QINV;    /* low 32 bits * QINV,
                                         mod 2^32 */
4     t = (a - (int64_t)t * Q) >> 32;    /* arithmetic right
                                         shift */
5     return t;
6 }
```

Listing 6.1: Signed Montgomery reduction (C reference)

6.3.2 Mathematical Analysis

Let $a \in \mathbb{Z}$ with $|a| < q \cdot 2^{15}$ (the working range for NTT inputs). Define:

$$m = \left((a \bmod 2^{32}) \cdot \text{QINV} \right) \bmod 2^{32} \in [0, 2^{32}), \quad (6.5)$$

$$u = a - m \cdot q. \quad (6.6)$$

Lemma 6.2 (Divisibility). $u \equiv 0 \pmod{2^{32}}$.

Proof. We have $m \equiv a \cdot \text{QINV} \pmod{2^{32}}$. Then $m \cdot q \equiv a \cdot \text{QINV} \cdot q \pmod{2^{32}}$. Since $\text{QINV} = q^{-1} \bmod R$, we get $\text{QINV} \cdot q \equiv 1 \pmod{2^{32}}$, so $m \cdot q \equiv a \pmod{2^{32}}$, hence $u = a - mq \equiv 0 \pmod{2^{32}}$. $\square$

Defining $t = u/2^{32}$ (exact integer division by Lemma 6.2), we have:

Theorem 6.3 (Montgomery reduction correctness). *Let $a \in \mathbb{Z}$ with $|a| < q \cdot 2^{15}$. Then:*

1. $t \equiv a \cdot R^{-1} \pmod{q}$, where $t = u/R$ and $u = a - m \cdot q$.
2. $-q \leq t < q$.

Proof. For Part (1): Since $u = a - mq$, we have $t = u/R = (a - mq)/R$, so $t \cdot R = a - mq \equiv a \pmod{q}$, thus $t \equiv a \cdot R^{-1} \pmod{q}$.

For Part (2), the checked proof does not use the unsigned representative $m \in [0, R)$ directly. It models the C cast to `int32_t` by a signed residue in $[-R/2, R/2)$ and proves the range for the signed REDC operator under the admissible precondition $-Rq/2 \leq a < Rq/2$. This gives the machine-checked bound $-q \leq t < q$. The simpler unsigned estimate only proves a weaker interval and is not sufficient for the stated result. $\square$

6.3.3 EasyCrypt Formalisation

The EasyCrypt formalisation of Montgomery reduction appears in `Fq.ec` and `Montgomery.ec`.

```

1 op SREDC (a: int) : int =
2   let u = smod (a * qinv * R) (R^2) in

```

```

3   let t = smod (a - u %/ R * q) (R^2) in
4       smod (t %/ R %% (R^2)) R.
5
6   lemma SREDCp_corr (a : int):
7       0 < q < R %/ 2 =>
8       -R %/ 2 * q <= a < R %/ 2 * q =>
9       -q <= SREDC a < q /\
10      SREDC a %% q = (a * Rinv) %% q.

```

Listing 6.2: Signed REDC correctness in `Montgomery.ec`

The key lemma `SREDCp_corr` is proved in two parts. The congruence part ($t \equiv aR^{-1} \pmod{q}$) follows from Lemma 6.2. The bound part uses the signed representative of the low word and the concrete values of q and $R = 2^{32}$ to establish $-q \leq t < q$ via SMT. The Jasmin procedure is connected to this operator by the Hoare lemma `montgomery_reduce_corr_h` in `Fq.ec`.

6.4 Montgomery Multiplication (`fqmul`)

Building on `montgomery_reduce`, the `fqmul` operation computes the product of two elements in $\mathbb{Z}_q$ in Montgomery form:

```

1   op fqmul (a b : int) : int = montgomery_reduce (a * b).

```

Listing 6.3: `fqmul` specification

Theorem 6.4 (*fqmul correctness*). *For all $a, b \in \mathbb{Z}$ whose product lies in the signed REDC admissible range $-Rq/2 \leq ab < Rq/2$:*

1. $\text{fqmul}(a, b) \equiv a \cdot b \cdot R^{-1} \pmod{q}$.
2. $|\text{fqmul}(a, b)| < q$.

Proof. Directly from Theorem 6.3 with the REDC input $a \leftarrow a \cdot b$. The refinement proof discharges the concrete side conditions for the NTT cases, for example when a 16-bit twiddle factor is multiplied by a coefficient bounded by the current stage invariant. $\square$

6.5 Twiddle Factor Multiplication in the NTT Butterfly

In the NTT butterfly, each coefficient $a[j + \ell]$ is multiplied by the twiddle factor **zeta**, which is stored in Montgomery form (i.e., as $\zeta \cdot R \bmod q$). The product passed to `montgomery_reduce` is thus $(\zeta \cdot R) \cdot a[j + \ell]$, and the output is:

$$\text{montgomery_reduce}(\zeta \cdot R \cdot a[j + \ell]) \equiv \zeta \cdot R \cdot a[j + \ell] \cdot R^{-1} = \zeta \cdot a[j + \ell] \pmod{q}.$$

This is exactly the required butterfly step: multiplication by the bare twiddle factor ζ . The Montgomery representation of the twiddle factors thus removes the need for any explicit conversion, at the cost of storing the twiddle table in pre-multiplied form.

Lemma 6.5 (Twiddle factor correctness). *Let $\zeta_k = \text{zetass}[k]$ (stored as $\zeta_0^{\text{brevss}(k)} \cdot R \bmod q$), and let a satisfy the signed coefficient bound required at the current stage. Then:*

$$\text{montgomery_reduce}(\zeta_k \cdot a) \equiv \zeta_0^{\text{brevss}(k)} \cdot a \pmod{q}$$

and $|\text{montgomery_reduce}(\zeta_k \cdot a)| < q$.

6.6 Bound Tracking Through NTT Stages

Theorem 6.6 (Coefficient bounds through forward NTT). *Let the concrete input array satisfy the checked 16-bit signed word bound `poly_repr_bound rp p 16`. After the forward NTT, the output satisfies the checked 24-bit signed word bound `poly_repr_bound res (full_ntt p) 24`.*

Proof. The checked proof in `RefJasminNTT.ec` uses stage-indexed bit-width bounds rather than the false recurrence $|a^{(s)}[i]| < 2^{15+s}$. For the forward loop, the function `fwd_stage_bound` maps

$$128, 64, 32, 16, 8, 4, 2, 1, 0 \quad \mapsto \quad 16, 17, 18, 19, 20, 21, 22, 23, 24.$$

Inner-loop and middle-loop invariants use `fwd_inner_bound` and `fwd_middle_bound` to mark exactly which cells have advanced to the next bit-width. The final EasyCrypt

theorem is `poly_ntt_core_ref`, whose postcondition is the 24-bit `poly_repr_bound`. □

Corollary 6.7 (Bound for inverse NTT). *The checked inverse refinement theorem assumes a 16-bit signed input bound and proves a 16-bit signed output bound for the Montgomery-form inverse result:*

$$\begin{aligned} & \text{poly_repr_bound } \text{rp } p \ 16 \\ & \implies \text{poly_repr_bound } \text{res } (\text{array256_mont}(\text{full_invntt } p)) \ 16. \end{aligned}$$

These bounds are the key invariants verified in `RefJasminNTT.ec` and summarised in Table 6.1.

Table 6.1: Coefficient bounds at each layer of the verification

Stage	Variable	Bound
Input to forward NTT	$a[i]$	signed 16-bit bound
After forward NTT	$\hat{a}[i]$	signed 24-bit bound
Input to inverse NTT	$\hat{a}[i]$	signed 16-bit bound required by theorem
After inverse NTT	$a'[i] \cdot R$	signed 16-bit bound
Montgomery reduced	$a'[i] \cdot R \bmod q$	signed representative in $[-q, q)$

Chapter 7

Imperative NTT Specification

7.1 Overview of NTT_Fq.ec

The file `NTT_Fq.ec` (988 lines) is the central algorithmic specification of this dissertation. It defines **EasyCrypt** procedures that faithfully model the forward and inverse NTT algorithms over field elements, separating the algorithmic butterfly structure from the machine-word Montgomery arithmetic proved in the refinement layer. The file serves two purposes:

1. It provides an *algorithmically correct* specification against which the **Jasmin** implementation can be formally compared.
2. It provides the *loop-level interface* to the abstract spectral specification in `NTTFullSpec.ec`.

7.2 Twiddle Factor Table

The forward NTT uses the constant array `zetas : coeff Array256.t` defined in `NTT_Fq.ec`. This is the abstract field table: `zetas[k]` is the bare field element $\zeta_0^{\text{brev}_8(k)}$, not the concrete Montgomery word stored in C/Jasmin. The concrete table is related to it by `array256_mont`. In particular, the abstract entry `zetas[0]` is 1, while the concrete C/Jasmin slot 0 is padding and is never read by the forward loop.

Lemma 7.1 (Twiddle factor definition). *For all $k \in [0, 256)$:*

$$\text{zetas}[k] = \zeta_0^{\text{brev}_8(k)},$$

where $\zeta_0 = 426$ and $R = 2^{32}$.

The Montgomery-word connection to the implementation table is proved separately by the `jzetas_get` and `jzetas_get_cancel` lemmas.

7.3 Forward NTT Procedure

7.3.1 Specification

The forward NTT is specified as the `EasyCrypt` procedure `NTT.ntt`:

```
1 module NTT = {
2   proc ntt(r : coeff Array256.t,
3         zetas : coeff Array256.t) : coeff Array256.t =
4     {
5       var len, start, j, zetasctr;
6       var t, zeta_;
7
8       zetasctr <- 0;
9       len <- 128;
10      while (1 <= len) {
11        start <- 0;
12        while (start < 256) {
13          zetasctr <- zetasctr + 1;
14          zeta_ <- zetas.[zetasctr];
15          j <- start;
16          while (j < start + len) {
17            t <- zeta_ * r.[j + len];
18            r.[j + len] <- r.[j] + (-t);
19            r.[j] <- r.[j] + t;
20            j <- j + 1;
21          }
22          start <- j + len;
23        }
24        len <- len %/ 2;
25      }
26      return r;
27    }.
}
```

Listing 7.1: Forward NTT specification in `NTT_Fq.ec`

7.3.2 Loop Structure Analysis

The three-level nested loop has the following structure:

Outer loop `while (len > 0)`: iterates $\ell \in \{128, 64, 32, 16, 8, 4, 2, 1\}$, corresponding to 8 NTT stages.

Middle loop `while (start < 256)`: for each stage, iterates over the $256/(2\ell)$ butterfly groups. At stage s , there are 2^s groups.

Inner loop `while (j < start + len)`: performs ℓ butterfly operations within each group, all using the same twiddle factor `zeta_ = zetas[zetasctr]`.

The twiddle counter `zetasctr` advances once per group (per iteration of the middle loop), consuming a total of $\sum_{s=0}^7 2^s = 255$ twiddle factors.

7.3.3 Loop Invariants

Definition 7.2 (Outer loop invariant I_{out}). After completing all stages with $\ell' > \ell$:

1. `zetasctr = fwd_zbase( $\ell$ )`, where `fwd_zbase` maps $128, 64, 32, \dots, 1$ to $0, 1, 3, \dots, 127$.
2. For all $i \in [0, 256)$, the coefficient $a[i]$ is partially transformed: it equals the sum of the appropriate butterfly outputs through stage $8 - \log_2(\ell)$.
3. For all $i \in [0, 256)$, the concrete word at index i satisfies the stage bit-width `fwd_stage_bound` applied to ℓ .

Definition 7.3 (Middle loop invariant I_{mid}). After completing all groups with $start' < start$ in stage s :

1. Indices in $[0, start)$ have been processed by stage s .
2. `zetasctr = fwd_zbase( $\ell$ ) + start/( $2\ell$ )`: the twiddle counter accounts for completed groups. The next concrete read is therefore at index `zetasctr + 1`.

3. Bounds are maintained as in I_{out} .

Definition 7.4 (Inner loop invariant I_{in}). After completing $j - \text{start}$ butterflies in group $[\text{start}, \text{start} + 2\ell)$:

1. Indices in $[\text{start}, j)$ and $[\text{start} + \ell, j + \ell)$ have been updated by the butterfly with factor $\zeta = \text{zetas}[\text{zetasctr}]$.
2. Indices in $[j, \text{start} + \ell)$ are unchanged from before this stage.
3. Bounds: updated indices use the next stage bit-width, as encoded by `fwd_inner_bound`; untouched indices retain the current `fwd_stage_bound` for ℓ .

7.3.4 Termination

Lemma 7.5 (NTT termination). *The procedure `NTT.ntt` terminates for any input array.*

Proof. Each outer loop iteration halves `len`, so after at most 8 iterations, `len` = 0 and the loop exits. For each fixed `len`, the middle loop increments `start` by $2 \cdot \text{len}$, so it terminates after at most $256/(2 \cdot \text{len})$ iterations. For each fixed `start`, the inner loop increments `j` by 1, terminating after exactly `len` iterations.

The variant functions are:

- Outer: `len` (decreases each iteration).
- Middle: $256 - \text{start}$ (decreases by $2 \cdot \text{len}$ each iteration).
- Inner: $\text{start} + \text{len} - j$ (decreases by 1 each iteration).

□

7.4 Inverse NTT Procedure

7.4.1 Specification

The inverse NTT is specified as the EasyCrypt procedure `NTT.invntt`:

```

1  module NTT = {
2      (* ... ntt as above ... *)
3
4      proc invntt(r : coeff Array256.t,
5                  zetas_inv : coeff Array256.t) : coeff
6                  Array256.t = {
7          var len, start, j, zetasctr;
8          var t, zeta_;
9
10         zetasctr <- 0;
11         len <- 1;
12         while (len < 256) {
13             start <- 0;
14             while (start < 256) {
15                 zeta_ <- zetas_inv.[zetasctr];
16                 zetasctr <- zetasctr + 1;
17                 j <- start;
18                 while (j < start + len) {
19                     t <- r.[j];
20                     r.[j] <- t + r.[j + len];
21                     r.[j + len] <- t + (-r.[j + len]);
22                     r.[j + len] <- zeta_ * r.[j + len];
23                     j <- j + 1;
24                 }
25                 start <- j + len;
26             }
27             len <- len * 2;
28         }
29         j <- 0;
30         while (j < 256) {
31             r.[j] <- r.[j] * zetas_inv.[255];
32             j <- j + 1;
33         }
34         return r;
35     }.

```

Listing 7.2: Inverse NTT specification in NTT_Fq.ec

7.4.2 Differences from the Forward NTT

The inverse NTT differs from the forward NTT in the following ways:

1. The outer loop runs ℓ from 1 to 128 (increasing, rather than decreasing from 128 to 1).
2. The abstract inverse table is read at indices $0, \dots, 254$ during the butterfly loops.
3. The butterfly is “gentle-side-up”: the update to $a[j + \ell]$ multiplies the difference by the inverse twiddle, while the update to $a[j]$ is a plain addition.
4. An extra final loop scales every coefficient by `zetas_inv[255]`, which is the field element 256^{-1} in `NTT_Fq.ec`. The concrete Jasmin table stores the corresponding Montgomery word -29720 .

7.4.3 Final Scaling

Lemma 7.6 (Final scaling correctness). *Let $a \in \mathbb{F}_q^{256}$ be the array after the main inverse NTT loops. In the abstract imperative specification, after the final scaling loop:*

$$a'[i] = a[i] \cdot 256^{-1}.$$

Proof. The checked lemma `zetas_inv_255` establishes `zetas_inv[255] = scale255`, where `scale255 = inv (incoeff 256)`. Thus the field-level loop multiplies by 256^{-1} . At the concrete Jasmin layer, the word `jzetas_inv[255]` is -29720 , and

$$-29720 \equiv 256^{-1}R^2 \pmod{q}.$$

One Montgomery multiplication by this word removes one factor of R , producing the representation relation `array256_mont` for the final inverse output. $\square$

7.5 Round-Trip Consequence

Proposition 7.7 (Mathematical round-trip consequence). *At the abstract field level, the forward and inverse spectral maps satisfy:*

$$\text{INTT}(\text{NTT}(a)) = a.$$

Proof sketch. By composition of correctness lemmas:

1. $\text{NTT}.\text{ntt}(a) = \hat{a}$ where $\hat{a}[i] = \text{NTT}(a)[i]$ (from the forward NTT correctness, proved in Chapter 8).
2. $\text{NTT}.\text{invntt}(\hat{a}) = a'$ where $a'[i] = \text{INTT}(\hat{a})[i]$ (from the inverse NTT correctness).
3. $\text{INTT}(\text{NTT}(a)) = a$ by the mathematical orthogonality argument.
4. The concrete Jasmin theorem adds the `array256_mont` representation relation, so the exported inverse output is stored in Montgomery form.

The checked EasyCrypt development proves the forward and inverse Hoare statements used in the first two steps. The final abstract identity above is stated as a mathematical consequence in the dissertation; the checked files do not introduce an additional one-procedure in-place pipeline theorem or a standalone checked round-trip lemma. $\square$

7.6 Twiddle Factor Synchronisation

A subtle but critical aspect of the imperative specification is the *twiddle factor synchronisation*: the counter k must be in exact correspondence with the current loop indices (ℓ, start) at every point in the algorithm. This invariant is captured formally as:

Lemma 7.8 (Twiddle counter synchronisation). *At any point during the execution of `NTT.ntt`, with current values ℓ_{cur} and $\text{start}_{\text{cur}}$:*

$$\text{zetascr} = \text{fwd_zbase}(\ell_{\text{cur}}) + \frac{\text{start}_{\text{cur}}}{2\ell_{\text{cur}}}.$$

This lemma is essential for connecting the imperative specification to the abstract spectral specification, where twiddle factors are indexed directly by the group number.

Chapter 8

Algebraic NTT Infrastructure

8.1 Overview of the Algebraic Files

The abstract spectral NTT specification lives in two files:

- `NTTFullSpec.ec` (160 lines) defines the abstract evaluation maps `full_ntt` and `full_invntt` and states the main correctness lemmas.
- `NTTFullAlgebra.ec` (3085 lines) develops the algebraic machinery needed to prove those lemmas: primitive root properties, bit-reversal identities, and butterfly-stage decompositions.

8.2 Primitive Root Properties

8.2.1 Existence and Uniqueness

The prime $q = 64513$ satisfies $q \equiv 1 \pmod{512}$. Therefore $\mathbb{F}_q^*$ contains an element of order 512, i.e., a primitive 512th root of unity.

Theorem 8.1 (Primitive root existence). *The element $\zeta_0 = 426 \in \mathbb{F}_q$ has multiplicative order exactly 512.*

Proof. We verify:

1. $\zeta_0^{512} \equiv 1 \pmod{64513}$ (direct computation).

2. For all $d \mid 512$ with $d < 512$: $\zeta_0^d \not\equiv 1 \pmod{64513}$. It suffices to check $d = 256$ and $d = 128$ (since $512 = 2^9$ and the order must divide 512).

Both computations are discharged by the `smt` tactic. $\square$

Corollary 8.2. $\zeta_0^{256} \equiv -1 \pmod{q}$ and $\zeta_0^{128} \not\equiv \pm 1 \pmod{q}$.

8.2.2 Orthogonality

The 512th roots of unity in $\mathbb{F}_q$ are $\{\zeta_0^k : k = 0, 1, \dots, 511\}$. The primitive roots are those ζ_0^k with $\gcd(k, 512) = 1$. The NTT formula uses the set of *odd* powers $\{\zeta_0^{2i+1} : i = 0, \dots, 255\}$, which are exactly the primitive roots.

Lemma 8.3 (Orthogonality relation). *For $i, j \in \{0, \dots, 255\}$ with $i \neq j$:*

$$\sum_{k=0}^{255} \zeta_0^{(2i+1)k} \cdot \zeta_0^{-(2j+1)k} = 0 \in \mathbb{F}_q.$$

For $i = j$, the sum equals 256.

Proof. For $i \neq j$: the sum is a geometric series with ratio $r = \zeta_0^{2(i-j)}$. Since ζ_0 has order 512, $r^{256} = 1$. Moreover $r \neq 1$, because $r = 1$ would imply $512 \mid 2(i-j)$, equivalently $256 \mid (i-j)$, impossible for distinct $i, j \in \{0, \dots, 255\}$. Thus $\sum_{k=0}^{255} r^k = (r^{256} - 1)/(r - 1) = 0$.

For $i = j$: each term is $\zeta_0^0 = 1$, and there are 256 terms. $\square$

8.3 Bit-Reversal Permutation

8.3.1 Definition

The bit-reversal permutation $\text{brev}_8 : \{0, \dots, 255\} \rightarrow \{0, \dots, 255\}$ reverses the 8 binary digits of its input:

$$\text{brev}_8(b_7b_6b_5b_4b_3b_2b_1b_0) = b_0b_1b_2b_3b_4b_5b_6b_7.$$

```
1 op bsrev (n k : int) : int =
2   bsrev8 k.    (* 8-bit bit reversal *)
```

```

3
4 op bsrev8 (k : int) : int =
5   let k0 = k %% 2 in
6   let k1 = (k %/ 2) %% 2 in
7   (* ... *)
8   let k7 = (k %/ 128) %% 2 in
9   k0 * 128 + k1 * 64 + k2 * 32 + k3 * 16 +
10  k4 * 8 + k5 * 4 + k6 * 2 + k7.

```

Listing 8.1: Bit-reversal definition in NTTFullAlgebra.ec

8.3.2 Key Properties

Lemma 8.4 (Bit-reversal involutivity). $\text{brev}_8(\text{brev}_8(k)) = k$ for all $k \in [0, 256)$.

Lemma 8.5 (Bit-reversal stage-index identities). *The checked proof uses several stage-specific identities, including:*

$$\text{brev}_8(2^k + i) = 2^{7-k} + \text{brev}_8(i) \quad (0 \leq k < 8, 0 \leq i < 2^k),$$

and

$$\text{brev}_8(2^k + ks) = (2 \cdot \text{brev}_k(ks) + 1) \cdot 2^{7-k} \quad (0 \leq ks < 2^k).$$

These identities, together with the address lemmas `bsrev_stage_pair_low`, `bsrev_stage_pair_high`, and `bsrev_zbase_index`, are what connect loop counters and array addresses to the spectral exponents. The proof does not rely on a generic “halving” formula for $\text{brev}_8(k \bmod \ell)$.

8.3.3 Bit-Reversal and Twiddle Factor Indexing

Lemma 8.6 (Twiddle factor ordering). *In the forward NTT outer loop with half-length $\ell = 2^{7-s}$, after completing g groups of stage s , the twiddle counter satisfies $k = 2^s + g$, and the twiddle factor is $\zeta_0^{\text{brev}_8(2^s+g)}$.*

Proof. By induction on s (outer loops) and g (groups within a stage). The key observation is that as the outer counter increments, the bit-reversal of the counter gives exactly the correct spectral index for the butterfly group. $\square$

8.4 Stage Decomposition Theorem

8.4.1 Partial NTT

Definition 8.7 (Segment-indexed partial NTT). The checked specification does not use a congruence-class definition $\text{NTT}_s(f)[i]$. Instead, `NTTFullSpec.ec` defines a segment-indexed partial sum:

$$\text{partial_ntt}(p, \ell, \text{start}, \text{bsj}) = \sum_{s=0}^{\ell-1} \zeta_0^{(2^{\text{start}+1})\text{brev}_8(s)} \cdot p[\text{brev}_8(\text{bsj} \cdot \ell + s)].$$

The full NTT specification states that for every $\text{start} \in [0, 256)$,

$$r[\text{brev}_8(\text{start})] = \text{partial_ntt}(p, 256, \text{start}, 0).$$

This formulation has the correct base cases and matches the way the loop proof grows the transformed segment length.

8.4.2 Butterfly Stage Lemma

Lemma 8.8 (Single-stage butterfly correctness). *For each forward stage, applying all butterflies at the current half-length transforms the collection of old segment-indexed partial sums into the collection for the next segment length:*

$$\text{partial_ntt}(\cdot, 2^k, \cdot, \cdot) \implies \text{partial_ntt}(\cdot, 2^{k+1}, \cdot, \cdot).$$

Proof. The proof splits each target partial sum into its low and high halves using `partial_ntt_split_low` and `partial_ntt_split_high`. The bit-reversal address lemmas show that the two source array cells used by the imperative butterfly are exactly the two old partial sums. The twiddle identity `zetastage` supplies the factor $\zeta_0^{\text{brev}_8(2^k+g)}$. The remaining equalities are field-ring rearrangements, discharged in the checked proof by the `ring` tactic and finite-sum rewrite lemmas. $\square$

8.4.3 Full NTT Correctness

Theorem 8.9 (Forward NTT correctness). *The imperative procedure `NTT.ntt` computes the full NTT:*

$$\forall a \in \mathbb{Z}_q^{256}. \text{NTT.ntt}(a) = \text{full_ntt}(a).$$

Proof. By induction on the outer loop iteration, using Lemma 8.8 as the inductive step. The loop invariant asserts that after s outer loop iterations, the array satisfies the appropriate segment-indexed `partial_ntt_spec` predicate for the input.

The base case is the length-1 segment specification, where each sum has one term. The inductive step applies Lemma 8.8 to double the segment length represented by the invariant. The final case is the length-256 segment specification, which is converted to `full_ntt` by `full_ntt_spec_imp` and Definition 4.1. $\square$

8.5 Inverse NTT Algebra

8.5.1 Abstract Inverse NTT

Definition 8.10 (Abstract inverse NTT, `full_invntt`).

$$\text{INTT}(p)[i] = 256^{-1} \sum_{j=0}^{255} p[j] \cdot \zeta_0^{-((2\text{brev}_8(j)+1) \cdot i)}$$

Proposition 8.11 (Pen-and-paper inverse identity). *`full_invntt(full_ntt(p)) = p` for all $p \in \mathbb{Z}_q^{256}$.*

Proof. Substituting the definition of `full_ntt`:

$$\begin{aligned}
\text{INTT}(\text{NTT}(p))[i] &= 256^{-1} \sum_{j=0}^{255} \text{NTT}(p)[j] \cdot \zeta_0^{-((2\text{brev}_8(j)+1)i)} \\
&= 256^{-1} \sum_{j=0}^{255} \left(\sum_{k=0}^{255} p[k] \cdot \zeta_0^{(2\text{brev}_8(j)+1)k} \right) \cdot \zeta_0^{-((2\text{brev}_8(j)+1)i)} \\
&= 256^{-1} \sum_{k=0}^{255} p[k] \sum_{j=0}^{255} \zeta_0^{(2\text{brev}_8(j)+1)(k-i)}.
\end{aligned}$$

For $k = i$: the inner sum is $\sum_{j=0}^{255} 1 = 256$. For $k \neq i$: the inner sum is 0 by a variant of Lemma 8.3 (substituting $m = \text{brev}_8(j)$ and using that brev_8 is a bijection on $[0, 256)$).

Thus $\text{INTT}(\text{NTT}(p))[i] = 256^{-1} \cdot p[i] \cdot 256 = p[i]$. $\square$

This proposition records the mathematical inverse identity used to interpret the transform. The checked EasyCrypt files in this artifact prove the forward and inverse loop refinements against `full_ntt` and `full_invntt` separately; they do not currently include a standalone machine-checked lemma named `full_invntt_full_ntt`.

8.6 The Critical Difference from MLKEM

Remark 8.12 (HAETAE vs. MLKEM exponent structure). In MLKEM, the twiddle factor for position (s, g) in the loop is $\zeta^{\text{brev}_8(2g+1)}$ (with a $2g+1$ structure), which corresponds to the *incomplete* 128-point transform used in that scheme. The algebra lemmas for MLKEM are stated in terms of brev_7 (7-bit reversal) rather than brev_8 .

In HAETAE, the twiddle factor is $\zeta^{\text{brev}_8(2^s+g)}$, where the bit-reversal acts on an index that includes the stage counter. This produces the *complete* 256-point negacyclic NTT. The corresponding algebra lemmas require brev_8 (8-bit reversal) and the orthogonality relation over all 256 primitive 512th roots of unity.

This distinction is the primary obstacle to reusing MLKEM’s EasyCrypt proofs, and is the motivation for developing `NTTFullAlgebra.ec` from scratch.

Chapter 9

Jasmin Implementation

9.1 Overview

The high-assurance implementation of the HAETAE NTT is written in the **Jasmin** programming language. Four source/support files collectively implement the forward NTT, inverse NTT, and base multiplication:

- `reduce.jinc`: Montgomery reduction and field multiplication.
- `zetas.jinc`: Precomputed twiddle factor table.
- `poly.jinc`: Forward and inverse NTT kernels.
- `hpoly.jazz`: Exported entry points callable from C.

The **Jasmin** source is compiled to x86-64 assembly (`hpoly.s`) and simultaneously extracted to an **EasyCrypt** module for formal verification. This chapter records two implementation shapes. The original direct-loop program is kept in the listings because it makes the correspondence with the C reference and the **EasyCrypt** loop invariants transparent. The current checked implementation used to regenerate `hpoly.s`, however, is the stage-specialised full-safe constant-time version described below: the public NTT schedule is split into fixed stage helpers, and the exported wrappers carry explicit regular and speculative constant-time contracts.

9.2 Montgomery Reduction in Jasmin

9.2.1 Source Code

```
1 param int JPRIME = 64513;
2 param int JQINV  = 940508161;
3
4 inline fn __montgomery_reduce(reg u64 a) -> reg u32 {
5     reg u64 t64;
6     reg u32 t, r;
7
8     t      = (32u)(a);           /* low 32 bits of a */
9     t      *= (u32)(JQINV);       /* t = (a mod 2^32) * QINV mod
10                                2^32 */
11     t64    = (64s)(t);          /* sign-extend to 64 bits */
12     t64    *= (u64)(JPRIME);     /* t64 = t * q (64-bit) */
13     t64    = a - t64;           /* t64 = a - t*q */
14     r      = (32u)(t64 >> 32);  /* r = (a - t*q) / 2^32 (
15                                arithmetic shift) */
16     return r;
17 }
```

Listing 9.1: Montgomery reduction in `reduce.jinc`

9.2.2 Jasmin Semantics

The Jasmin code uses explicit types to control the width and signedness of arithmetic:

- `(32u)(a)`: truncate 64-bit value to 32 bits (unsigned).
- `(u32)(JQINV)`: interpret the literal as a 32-bit unsigned value.
- `(64s)(t)`: sign-extend a 32-bit value to 64 bits.
- `(32u)(t64 >> 32)`: arithmetic right shift and truncate.

The `>>` operator on a signed 64-bit value in Jasmin performs an *arithmetic* right shift, matching the C behaviour for signed integers.

9.2.3 Correspondence to C

The Jasmin code directly implements the C reference in Listing 6.1:

- $(32u)(a) * (u32)(JQINV)$ corresponds to $(int32_t)a * (int32_t)QINV$.
- The sign extension $(64s)(t)$ followed by multiplication by $JPRIME$ corresponds to $(int64_t)t * Q$.
- The arithmetic right shift $\gg 32$ corresponds to $\gg 32$ on a signed value.

9.3 Field Multiplication

```

1 inline fn __fqmul(reg u32 a, reg u32 b) -> reg u32 {
2     reg u64 t64;
3     reg u32 r;
4
5     t64 = (64s)(a);
6     t64 *= (64s)(b);
7     r    = __montgomery_reduce(t64);
8     return r;
9 }

```

Listing 9.2: Field multiplication in `reduce.jinc`

This computes $a \cdot b \cdot R^{-1} \bmod q$. The refinement proof discharges the product-range side condition for the NTT cases, including 16-bit twiddle factors multiplied by coefficients bounded by the current stage invariant.

9.4 Forward NTT Kernel

9.4.1 Original Direct Source Code

```

1 inline fn _poly_ntt(reg mut ptr u32[256] rp) -> reg ptr
2     u32[256] {
3     reg u32 zeta, t;
4     reg int len, start, j, zetasctr;

```

```
5   zetasctr = 0;
6   len = 128;
7
8   while (len > 0) {
9       start = 0;
10      while (start < 256) {
11          zetasctr += 1;
12          zeta = (u32) jzetas[zetasctr];
13
14          j = start;
15          while (j < start + len) {
16              t = __fqmul(zeta, rp[j + len]);
17
18              rp[j + len] = rp[j] - t;
19              rp[j]      = rp[j] + t;
20              j += 1;
21          }
22          start = j + len;
23      }
24      len >>= 1;
25  }
26  return rp;
27 }
```

Listing 9.3: Original direct forward NTT kernel in `poly.jinc`

Listing 9.3 is the original direct implementation. It is retained as the readable algorithmic baseline: it exposes the same three nested loops as the C reference. In the current compiler-checked source, this public outer schedule is no longer represented by run-time variables `len`, `start`, and `zetasctr`. Instead, each stage is a separate helper with literal bounds and a literal twiddle-table base.

9.4.2 Structural Correspondence

The Jasmin kernel `_poly_ntt` has a direct structural correspondence to the C reference (Listing 3.1) and the EasyCrypt specification (Listing 7.1):

Aspect	C reference	Jasmin
Outer loop	<code>len = 128; len > 0; len >>= 1</code>	<code>while (len > 0)</code>
Middle loop	<code>start < 256; start += 2*len</code>	equivalent <code>while</code>
Inner loop	<code>j = start; j < start+len; j++</code>	equivalent <code>while</code>
Twiddle	<code>zetas[k++]</code>	<code>zetasctr += 1;</code> <code>jzetas[zetasctr]</code>
Butterfly	<code>mont_reduce(zeta * a[j+len])</code>	same

The key difference is the use of explicit type casts in `Jasmin` to manage word widths.

9.4.3 Array Access Pattern

In `Jasmin`, the global constant table `jzetas` is accessed as a `u32` array. The entry at index k is the Montgomery form of the k -th twiddle factor. The access pattern increments `zetasctr` before each read, so the forward kernel consumes `jzetas[1]` through `jzetas[255]`; `jzetas[0]` is padding.

9.4.4 Full-Safe Stage-Specialised Forward Kernel

The full-safe implementation keeps the same butterfly operations and twiddle order, but makes every stage bound explicit to the `Jasmin` safety checker. For the first forward stage, the checker sees the literal facts `block < 1`, `j < 128`, and `offset = idx + 128`; hence both `rp[idx]` and `rp[offset]` are statically within the 256-word polynomial. The remaining helpers instantiate the same pattern for 64, 32, 16, 8, 4, 2, 1.

```

1 fn _poly_ntt_stage_128(reg mut ptr u32[256] rp)
2   -> reg ptr u32[256] {
3     reg ptr u32[256] zetasp;
4     reg ui64 block, j, start, idx, offset;
5     reg u32 zeta, t, s, coeff;
6
7     zetasp = jzetas;
8     block = 0;
9     while (block < 1) {
10      zeta = zetasp[1 + block];
11      start = block;
12      start <=< 8;
13      j = 0;

```

```

14     while (j < 128) {
15         idx = j;
16         idx += start;
17         s = rp[idx];
18         offset = idx;
19         offset += 128;
20         coeff = rp[offset];
21         t = __fqmul(zeta, coeff);
22         coeff = s;
23         coeff -= t;
24         rp[offset] = coeff;
25         s += t;
26         rp[idx] = s;
27         j += 1;
28     }
29     block += 1;
30 }
31 return rp;
32 }
33
34 fn _poly_ntt(reg mut ptr u32[256] rp) -> reg ptr u32[256]
35 {
36     rp = _poly_ntt_stage_128(rp);
37     rp = _poly_ntt_stage_64(rp);
38     rp = _poly_ntt_stage_32(rp);
39     rp = _poly_ntt_stage_16(rp);
40     rp = _poly_ntt_stage_8(rp);
41     rp = _poly_ntt_stage_4(rp);
42     rp = _poly_ntt_stage_2(rp);
43     rp = _poly_ntt_stage_1(rp);
44     return rp;
45 }

```

Listing 9.4: Current full-safe forward NTT stage shape in `poly.jinc`

The transformation is proof-preserving at the algorithmic level: the sequence of helper calls is the fixed unrolling of the original public `len` schedule. The implementation change is motivated by compiler verification, not by a change in the mathematical NTT.

9.5 Inverse NTT Kernel

```

1  inline fn _poly_invntt(reg mut ptr u32[256] rp) -> reg ptr
    u32[256] {
2      reg u32 zeta, t, coeff;
3      reg int len, start, j, zetasctr;
4
5      zetasctr = 0;
6      len = 1;
7
8      while (len < 256) {
9          start = 0;
10         while (start < 256) {
11             zeta = (u32) jzetas_inv[zetasctr];
12             zetasctr += 1;
13
14             j = start;
15             while (j < start + len) {
16                 t = rp[j];
17                 coeff = rp[j + len];
18                 rp[j] = t + coeff;
19                 rp[j + len] = __fqmul(zeta, t - coeff);
20
21                 j += 1;
22             }
23             start = j + len;
24         }
25         len <<= 1;
26     }
27
28     /* Final scaling */
29     zeta = (u32) jzetas_inv[255];
30     j = 0;
31     while (j < 256) {
32         rp[j] = __fqmul(zeta, rp[j]);
33         j += 1;
34     }
35     return rp;
36 }

```

Listing 9.5: Original direct inverse NTT kernel in `poly.jinc` (condensed)

The inverse butterfly loop consumes `jzetas_inv[0..254]`. The final scaling loop is the only place where `jzetas_inv[255]` is read. Listing 9.5 is likewise retained as the original direct-loop form. The current checked source splits the inverse transform into helpers `_poly_invntt_stage_1`, `_poly_invntt_stage_2`, ..., `_poly_invntt_stage_128`, followed by `_poly_invntt_scale`. The helper sequence fixes the twiddle bases 0, 128, 192, 224, 240, 248, 252, 254, and the scale helper reads `jzetas_inv[255]`.

9.5.1 Inverse Butterfly Structure

The inverse butterfly in Jasmin computes:

$$\begin{aligned} t &\leftarrow \text{rp}[j] \\ \text{rp}[j] &\leftarrow t + \text{rp}[j + \ell] \\ \text{rp}[j + \ell] &\leftarrow \text{mont}(\zeta \cdot (t - \text{rp}[j + \ell])) \end{aligned}$$

This is the inverse (Gentleman–Sande) butterfly. It is the algebraic inverse of the Cooley–Tukey butterfly: if the forward butterfly computes $(a + b, a - b)$ with twiddle w applied to the lower output, the inverse butterfly computes $(a + b, w(a - b))$ and is composed with the twiddle application on the upper output in the forward case.

```
1 fn _poly_invntt(reg mut ptr u32[256] rp) -> reg ptr u32
   [256] {
2   rp = _poly_invntt_stage_1(rp);
3   rp = _poly_invntt_stage_2(rp);
4   rp = _poly_invntt_stage_4(rp);
5   rp = _poly_invntt_stage_8(rp);
6   rp = _poly_invntt_stage_16(rp);
7   rp = _poly_invntt_stage_32(rp);
8   rp = _poly_invntt_stage_64(rp);
9   rp = _poly_invntt_stage_128(rp);
10  rp = _poly_invntt_scale(rp);
11  return rp;
12 }
```

Listing 9.6: Current full-safe inverse NTT schedule in `poly.jinc`

9.6 Exported Entry Points

```

1 export fn poly_ntt_jazz(reg mut ptr u32[256] rp)
2   -> reg ptr u32[256] {
3     rp = _poly_ntt(rp);
4     return rp;
5   }
6
7 export fn poly_invntt_jazz(reg mut ptr u32[256] rp)
8   -> reg ptr u32[256] {
9     rp = _poly_invntt(rp);
10    return rp;
11  }
12
13 export fn poly_basemul_jazz(
14   reg mut ptr u32[256] rp,
15   reg ptr u32[256] ap,
16   reg ptr u32[256] bp)
17   -> reg ptr u32[256] {
18     rp = _poly_basemul(rp, ap, bp);
19     return rp;
20  }

```

Listing 9.7: Original exported functions in `hpoly.jazz`

Listing 9.7 is the original wrapper form. The current full-safe constant-time wrapper form adds explicit contracts for both ordinary constant-time checking and speculative constant-time checking. The `#init_msf()` call initializes the microarchitectural speculation fence state required by `jasmin-ct -speculative`; in the generated assembly it emits an `lfence` at the public entry point.

```

1 #[ct = "rp -> rp"]
2 #[sct = "{ ptr: transient, val: secret } -> { ptr: public,
3   val: secret }"]
3 export

```

```

4 fn poly_ntt_jazz(reg mut ptr u32[256] rp) -> reg ptr u32
   [256] {
5   _ = #init_msf();
6   rp = _poly_ntt(rp);
7   return rp;
8 }
9
10 #[ct = "rp -> rp"]
11 #[sct = "{ ptr: transient, val: secret } -> { ptr: public,
   val: secret }"]
12 export
13 fn poly_invntt_jazz(reg mut ptr u32[256] rp) -> reg ptr
   u32[256] {
14   _ = #init_msf();
15   rp = _poly_invntt(rp);
16   return rp;
17 }
18
19 #[ct = "rp × ap × bp -> {ap, bp, rp}"]
20 #[sct = "
21 { ptr: transient, val: secret } ×
22 { ptr: transient, val: secret } ×
23 { ptr: transient, val: secret } ->
24 { ptr: public, val: secret }
25 "]
26 export
27 fn poly_basemul_jazz(reg mut ptr u32[256] rp,
28                     reg const ptr u32[256] ap bp) -> reg
   ptr u32[256] {
29   _ = #init_msf();
30   rp = _poly_basemul(rp, ap, bp);
31   return rp;
32 }

```

Listing 9.8: Current constant-time exported wrappers in `hpoly.jazz`

The `export` keyword triggers both assembly generation and `EasyCrypt` extraction. The ABI of `poly_ntt_jazz` takes a pointer to a 256-word array in a register and returns the same pointer after in-place modification. The argument names follow the conventional pointer suffix `p`. Thus `rp` is the result-polynomial pointer.

It is mutable and is used in-place by `poly_ntt_jazz` and `poly_invntt_jazz`; in `poly_basemul_jazz` it names the destination array that receives the coefficientwise products. The names `ap` and `bp` are the two read-only input-polynomial pointers for the base-multiplication operands a and b . All three pointers range over arrays of `u32[256]` coefficients, but only `rp` is declared `mut`; the current checked wrapper declares `ap` and `bp` as `const` to make their read-only role explicit.

9.7 Safety and Constant-Time Verification

The Jasmin verification obligations attached to this chapter are not only functional. Functional refinement, which is carried out later in `EasyCrypt`, proves that the extracted procedures implement the mathematical NTT. The compiler-side checks discussed here prove two lower-level properties that must hold before the functional statement can be trusted as an implementation statement: the program must be *safe*, so that the low-level execution is defined, and it must be *constant time*, so that its control flow and memory-access pattern do not depend on polynomial coefficients. The account below follows the local Jasmin checkout at `$HOME/jasmin`: the safety checker is implemented under `compiler/safetylib`, the regular constant-time checker in `compiler/src/ct_checker_forward.ml`, the speculative checker in `compiler/src/sct_checker_forward.ml`, and the command-line entry point for the latter two in `compiler/entry/jasmin_ct.ml`. The conceptual background is the CCS 2017 Jasmin paper [Alm+17], which presents the language as a way to keep assembly-level control while making source-level safety and timing analyses tractable. That paper describes the original source-analysis pipeline in terms of two translations: a functional embedding used to prove safety, and a constant-time instrumentation whose product program reduces timing security to a safety proof. The local tooling used in this dissertation has evolved, but the proof principles are the same: first establish that the execution is defined, then establish that the observable trace is independent of secrets, and finally rely on the Jasmin compiler’s preservation argument to carry those source-level properties to the generated assembly.

9.7.1 What Safety Means for This Code

Safety is the definedness layer of the proof. The Jasmin type system gives each pointer and array an element width and a length, for example `reg mut ptr u32[256] rp`. The safety checker then verifies that every operation which could be undefined at the

assembly level is executed only under conditions that make it valid. For this scalar NTT implementation the important safety obligations are:

- **Valid memory accesses.** Every read and write through `rp`, `ap`, or `bp` must be inside the corresponding 256-word polynomial object. Every access to `jzetas` or `jzetas_inv` must be inside the 256-entry global table.
- **Alignment and access width.** A `u32` access must be made at an address that is valid for a 32-bit load or store. In the source this is expressed by typed `u32[256]` pointers rather than by byte-level address arithmetic.
- **Defined control flow.** Loops must have public, bounded schedules. The checker reports termination obligations separately from memory obligations; in this implementation all loops count over fixed stage constants.
- **Defined use of primitive operations.** Word arithmetic itself is not C-style undefined arithmetic: **Jasmin** word operations have fixed bit-vector semantics. The safety proof therefore does not need to prove absence of signed overflow in Montgomery arithmetic. It does need to prove that primitive operations are used at valid types and widths and that memory operands are valid.

In the CCS 2017 presentation, this safety condition is explained through a near one-to-one functional embedding of **Jasmin** into a verification language: fixed-size **Jasmin** arrays become fixed-size sequences, memory is modeled as byte blocks, and every memory read or write is guarded by validity assertions for the addressed byte range [Alm+17]. The modern `-checksafety` command no longer needs to be described as a Dafny proof in order to use it in this artifact, but the generated obligations have the same shape. A safety success means that every array index, memory address, shift amount, and other potentially failing operation has been proved valid under the function’s typed calling convention and loop invariants.

The present implementation uses a deliberately small fragment of that memory model. The source code in `poly.jinc` has no explicit raw memory access of the form `[p + e]` and no source-level stack array. Its observable memory operations are typed array/pointer operations: `rp[idx]`, `rp[offset]`, `ap[i]`, `bp[i]`, and reads from the global tables `jzetas` and `jzetas_inv`. Thus the abstract byte-block obligations from CCS17 specialize here to simple word-array obligations: each caller-provided polynomial pointer must designate a valid 256-word `u32` region, and every computed index must lie in $[0, 256)$. This is why the implementation avoids pointer arithmetic

in the source even though the generated x86 code necessarily lowers the same facts to base-plus-scaled-index addressing.

The mathematical safety argument for the original direct-loop code is simple. For a forward stage with length ℓ , the inner loop reads and writes `rp[j]` and `rp[j +  $\ell$ ]`, with

$$j \in [\text{start}, \text{start} + \ell) \quad \text{and} \quad j + \ell \in [\text{start} + \ell, \text{start} + 2\ell).$$

The middle loop advances `start` by 2ℓ , so the two half-blocks are always contained in one interval $[\text{start}, \text{start} + 2\ell) \subseteq [0, 256)$. The inverse butterfly has the same address shape. The twiddle-table argument is also fixed: the forward transform consumes `jzetas[1]` through `jzetas[255]`; the inverse butterfly stages consume `jzetas_inv[0]` through `jzetas_inv[254]`; and the final scaling loop consumes `jzetas_inv[255]`. Base multiplication has the simplest safety shape. The loop counter `i` ranges from 0 to 255, each iteration reads exactly `ap[i]` and `bp[i]`, and it writes exactly `rp[i]`. The declarations in `hpoly.jazz` mark `ap` and `bp` as `const` pointers, so the safety proof for this routine also matches the intended data-flow role: the two source polynomials are read, and the result polynomial is the only destination.

The stage-specialised implementation exists to make these same facts directly visible to the automatic checker. A human can infer from the direct program that the triple $(\text{len}, \text{start}, j)$ maintains the range invariant above. An abstract interpreter is deliberately more local and more conservative: it must discover enough arithmetic relationships between mutable loop variables to prove every array access. The full-safe source therefore replaces the public run-time stage variable `len` by eight helper functions with literal lengths 128, 64, 32, 16, 8, 4, 2, 1. In a helper with length ℓ , the code has the schematic form

$$\text{block} < \frac{256}{2\ell}, \quad j < \ell, \quad \text{start} = \text{block} \cdot 2\ell, \quad \text{offset} = \text{start} + j + \ell.$$

Consequently the checker only needs local arithmetic to derive

$$0 \leq \text{idx} < 256 \quad \text{and} \quad 0 \leq \text{offset} < 256.$$

The first forward helper shown in Listing 9.4 is the extreme case: `block` < 1, `j` < 128, and `offset` = `j` + 128. Later helpers instantiate the same proof with smaller ℓ and more blocks. The inverse helpers use the same address proof in the opposite stage order, and the final scaling helper has only the single index obligation $0 \leq j < 256$.

9.7.2 Regular Constant-Time Typing

The regular constant-time checker is a security type checker. Its lattice has public, secret, and polymorphic levels. Public data may influence observable timing behaviour; secret data may not. Polymorphic levels are used in function contracts so that a returned value can be described as depending on a particular input without forcing that input to be public. The checker then walks the program and enforces the following discipline.

The semantic property behind this type discipline is trace equality. The *Jasmin* paper instruments the operational semantics with a leakage trace recording the branches taken and the memory read/write operations performed during execution. A command is constant time when any two executions that agree on public inputs produce the same leakage trace, even if their secret state differs [Alm+17]. This is a termination-insensitive property, so safety is checked separately and first; the dissertation follows the same ordering by pairing `jasminc -checksafety` with `jasmin-ct`.

- Conditions of `if` statements, `while` guards, and loop bounds must type-check as public. Thus a secret coefficient cannot determine whether an instruction executes.
- Memory addresses and array indices must type-check as public. Thus a secret coefficient cannot determine which cache line is read or written.
- Operations classified by the architecture as not constant time may be used only on public operands. This is why the checker is parameterised by the architecture-specific predicate passed to `jasmin-ct`.
- The result of a memory load is treated conservatively as secret unless its source is known more precisely through the typed array discipline. This is exactly the desired default for cryptographic state.

This is a proof relative to the *Jasmin* x86 leakage model used by the checker. Following CCS17, the observable trace records branches and memory read/write operations, while instructions classified as constant-time by the architecture model do not add secret-dependent leakage obligations. This matters for the present code because Montgomery multiplication lowers to `imul` and `sar` instructions on coefficient-derived registers: the checker accepts these arithmetic operations under the selected x86

model, but it would reject a use of secret data in an operation that the model marks as non-constant-time or in any address/branch expression.

The CCS 2017 toolchain checked this property by inserting publicness annotations on control-flow conditions, memory accesses, and array accesses, then using a Boogie product program that executes two copies of the program in lockstep. Publicness assertions become relational equalities between the original and shadow executions; proving the product program safe proves that the two leakage traces coincide [Alm+17]. The local `jasmin-ct` checker used here is a lighter direct type checker, but it is enforcing the same relational idea syntactically: anything that contributes to the leakage trace must be public.

For the HAETAETNT, all data-dependent values are coefficient values or intermediate products inside `__fqmul` and `__montgomery_reduce`. Those values flow through additions, subtractions, casts, shifts, and multiplications, but they never flow into a branch condition, a loop bound, or an address expression. The schedule is fixed by the public stage constants. The table indices are also public: `1 + block`, `2 + block`, ..., and the inverse-stage bases are literal constants. Therefore the memory trace of the NTT is independent of the coefficients even though the loaded and stored words themselves are secret.

The same trace argument applies to `_poly_basemul`. The routine has one public loop counter `i`; on each iteration it reads `ap[i]` and `bp[i]`, computes `__fqmul(a, b)`, and writes the result to `rp[i]`. The loaded coefficients `a` and `b` are secret, and the Montgomery product is secret, but the only branch condition is `i < HAETAET_N` and the only addresses use the public index `i`. The generated assembly consequently contains the expected public-counter loop: loads from the two source bases at scale four, a store to the result base at the same public index, and a comparison of the loop counter with 256. This is exactly the CCS17 leakage discipline instantiated on the actual scalar base-multiplication code.

The exported contracts in Listing 9.8 record this policy at the ABI boundary. The contract `#[ct = "rp -> rp"]` says that the output of the in-place NTT has the same security dependence as its input polynomial. It does not say that the polynomial is public; it says that the function preserves the input's security level while the checker separately enforces public use of control and address expressions. For base multiplication, the contract `#[ct = "rp * ap * bp -> {ap, bp, rp}"]` records the analogous three-argument policy. Here the identifiers `rp`, `ap`, and `bp` are also used as named security levels in the `Jasmin` annotation syntax. They are chosen to

match the pointer variables: `rp` labels the result-pointer argument, while `ap` and `bp` label the two source-polynomial arguments. The result type `{ap, bp, rp}` is intentionally conservative: the returned pointer is allowed to carry dependence on all three input labels, even though the source body returns the destination pointer after writing all 256 cells. The important constant-time fact is separate from minimality of this return label: the checker still requires the loop index used in `ap[i]`, `bp[i]`, and `rp[i]` to be public, so secret coefficient values can affect only the words written to `rp`, not which locations are read or written.

9.7.3 Speculative Constant-Time Typing

The speculative checker strengthens the regular property against transient execution. In the source language this is expressed by the `sct` attribute. Unlike the regular `ct` contract, an `sct` type distinguishes the security of a pointer value from the security of the memory contents reachable through that pointer:

$$\{ \text{ptr}: \ell_p, \text{val}: \ell_v \}.$$

It also distinguishes ordinary execution from speculative execution. The special level `transient` is printed by the Jasmin checker for the pair (normal public, speculative secret). Thus an argument of type

$$\{ \text{ptr}: \text{transient}, \text{val}: \text{secret} \}$$

means: on the architecturally committed path, the pointer is public enough to be used as an address; during speculative execution, the checker treats it as potentially secret until a speculation barrier or hardening operation has established a public status. The memory values remain secret in both modes.

The instruction `#init_msf()` is the explicit boundary used by the exported wrappers. In the checker it initializes the Jasmin MSF state and normalises speculative security levels after the public entry point has been reached. In the x86-64 lowering used here, the corresponding exported assembly contains an `lfence`. The wrappers therefore have the shape

transient pointer input $\xrightarrow{\text{\#init_msf()}}$ public pointer schedule with secret values.

After that boundary, the same fixed-stage argument used for regular constant-time

applies, but now every array access is checked in the stronger speculative type environment. The output contract `{ ptr: public, val: secret }` states that returning the pointer does not leak secret information and that the polynomial contents are still secret. For `poly_basemul_jazz`, the three input arguments all have the same speculative type: each pointer is transient at entry and each pointed-to polynomial value is secret. After `#init_msf()`, the checker may use `rp`, `ap`, and `bp` as public address bases, while the values loaded from `ap[i]` and `bp[i]` remain secret and flow only into the stored coefficient `rp[i]`.

This distinction matters for an exported cryptographic routine. A regular constant-time proof rules out leaks from committed branches and committed addresses. Speculative constant-time additionally prevents the proof from silently assuming that a public-looking pointer or bound remains public under transient execution before the entry fence has taken effect. In this implementation there are no secret-dependent bounds to harden inside the polynomial kernels; the only required hardening point is the public wrapper entry.

9.7.4 Why Source-Level Checks Apply to Assembly

The *Jasmin* paper separates two questions that are easy to conflate. The first is whether the source program is safe and constant time. The second is whether compilation can invalidate those properties. *Jasmin*'s answer is to restrict the compiler to predictable transformations and prove semantic preservation in Coq, then argue that the compiler also preserves the constant-time leakage property [Alm+17]. In particular, the compiler does not perform the aggressive optimisations that are dangerous for constant-time code, such as replacing computations on secret values with table lookups or introducing case splits on secret-dependent ranges. Conditional operations are lowered to constant-time conditional instructions rather than to secret-dependent branches, and transformations such as loop unrolling or constant propagation remove only public, fixed control structure.

This point is central for the present artifact. The verification commands are run on `jasmin/hpoly.jazz`, not by manually auditing every generated assembly instruction. That is meaningful because the *Jasmin* compilation discipline is designed so that source-level public schedule and public address facts remain public in the assembly trace. The generated `hpoly.s` therefore inherits the fixed-stage NTT schedule: any additional loads or stores introduced by allocation are at locations determined by the public calling convention and public loop counters, not by coefficient values.

This can be seen directly in the generated scalar assembly. The exported wrappers begin with an `lfence` corresponding to `#init_msf()`. The base-multiplication loop lowers `ap[i]`, `bp[i]`, and `rp[i]` to indexed memory operands with the public loop counter as the scaled index; the loop branch compares that counter against the constant 256. Likewise, each NTT stage loop branches on public counters and uses addresses formed from the public base pointer, public stage constants, and public counters. Secret coefficients appear in arithmetic registers and in stored words, but not in branch predicates or address registers.

9.7.5 Instantiation for the HAETAET NTT

Combining the safety and timing arguments gives a precise invariant for every exported routine:

- **Public schedule.** The number of iterations in every loop is a function only of the fixed transform size 256 and the current stage length. It is independent of coefficient values.
- **Public addresses.** The address expressions for `rp[idx]`, `rp[offset]`, `ap[i]`, `bp[i]`, and the twiddle tables are affine expressions in public loop counters and literal stage constants.
- **Secret arithmetic.** Coefficients, products, Montgomery reductions, and butterfly sums are allowed to be secret. They affect stored values but never affect which instructions execute or which addresses are touched.
- **Defined accesses.** The same public affine expressions are bounded strongly enough to prove all array indices lie in $[0, 256)$.
- **Speculation boundary.** Each exported wrapper initializes the MSF state before calling the internal kernel, turning transient pointer inputs into public pointer schedules while preserving secret values.

The implementation style is therefore intentionally conservative. It avoids data-dependent conditional subtraction, table lookups indexed by coefficients, variable-length loops, and secret-dependent pointer arithmetic. Some of these patterns could be made secure with additional masking or hardening, but they would enlarge the trusted timing argument. The scalar helperized code keeps the checker’s proof

obligation identical to the informal cryptographic rule: only the public NTT schedule determines the trace.

This is the main reason the verification claim is persuasive rather than only a tool invocation. The desired statement is a two-run noninterference property: two calls that satisfy the same public ABI and pointer-validity preconditions, but contain different secret coefficient values, must have the same branch and memory-address trace. Safety supplies the defined-execution premise for those two runs; regular constant-time typing proves that every expression contributing to the trace is public; speculative constant-time typing proves that the exported entry boundary first converts transient pointer inputs into public address bases without making polynomial contents public; and the Jasmin compiler-preservation theorem carries those source facts to the generated assembly. In this implementation each premise can be inspected directly in the source: loop guards are public constants and counters, address expressions are affine in public counters and stage offsets, and coefficients appear only in arithmetic registers and stored words. The checker is therefore validating the same invariant a manual audit would look for, but it does so exhaustively over all stages, all loop iterations, and all exported routines.

9.7.6 Concrete Checks

The following checks were run on `jasmin/hpoly.jazz` after regenerating the source:

```

1  jasminc -checksafety -o /tmp/hpoly-safe.s jasmin/hpoly.
   jazz
2  poly_ntt_jazz:      No Safety Violation
3  poly_invntt_jazz: No Safety Violation
4  poly_basemul_jazz: No Safety Violation
5
6  jasmin-ct jasmin/hpoly.jazz
7  regular constant-time check: passed
8
9  jasmin-ct --speculative jasmin/hpoly.jazz
10 speculative constant-time check: passed

```

Listing 9.9: Compiler checks for the full-safe constant-time implementation

The three results should be read together. The safety result says that the compiler found no possible out-of-bounds, invalid, or misaligned execution of the exported

routines under the typed input-pointer assumptions. The regular constant-time result says that committed control flow and committed memory addresses are independent of secret coefficients. The speculative constant-time result says that this remains valid in the stronger transient model induced by the `sct` annotations and the `#init_msf()` boundary. Consequently, the checked assembly has a fixed instruction and memory-access schedule for all coefficient values, while the values carried by that schedule remain secret.

9.8 Generated Assembly

The compiled assembly `hpoly.s` contains straightforward x86-64 code. Key assembly features include:

- Register allocation: stage-local variables (`block`, `start`, `j`, `idx`, and `offset`) are assigned to general-purpose registers.
- Montgomery reduction uses `imul` (signed 64-bit multiply) and `sar` (arithmetic shift right) instructions.
- The exported wrappers contain an entry `lfence`, emitted by `#init_msf()`, to satisfy the speculative constant-time contract.
- No SIMD instructions are used in the scalar implementation; a vectorised (AVX2) version is possible but beyond the scope of this dissertation.

The correspondence between the `Jasmin` source, the generated assembly, and the `EasyCrypt` extraction is guaranteed by the `Jasmin` compiler correctness theorem [Alm+17], which is itself a machine-checked proof in the Coq proof assistant.

Chapter 10

Jasmin-to-EasyCrypt Refinement Proof

10.1 Overview of the Jasmin Refinement Proofs

The file `RefJasminNTT.ec` (2609 lines) bridges the gap between the original direct-loop `Jasmin` implementation and the abstract algorithmic specification. It is the most technically demanding proof file in the development, requiring simultaneous reasoning about:

1. Signed vs. unsigned word arithmetic.
2. Array representation correspondences.
3. Integer arithmetic bounds.
4. Equivalence of three-level nested loops.

The proof presentation in this chapter follows the original direct-loop program shown in Listings 9.3 and 9.5. The current full-safe `Jasmin` source described in Listing 9.4 and Listing 9.6 exposes the same public stage schedule as a composition of fixed helpers. Thus the algorithmic refinement argument is read stage by stage: each helper is the corresponding instance of the old outer-loop body, with the public `len` and twiddle-base values made literal for `jasminc`.

The current machine-checked full-safe artifact keeps the original `RefJasminNTT.ec` proof boundary as historical documentation and adds a separate `easycrypt-ct`

bridge for the compiler-safe source: `Hpoly_loop.ec` defines a proof-only direct-loop model with the same tables and multiplication routine as the extraction, `RefJasminNTTLoop.ec` reuses the direct-loop refinement proof against that model, and `CTLoopEquiv.ec` proves that the helperized `Hpoly_extract` procedures are equivalent to the proof-only loop. The final theorems in `NTTEndToEnd.ec` therefore target the full-safe `Jasmin` extraction, not only the old direct-loop presentation.

10.1.1 Full-Safe Constant-Time Bridge

The helperized source changes the control-flow shape seen by `jasminc` but not the public NTT schedule. The proof uses this fact as a small translation layer rather than duplicating the whole refinement:

1. **`Hpoly_loop.ec`**: a direct-loop EasyCrypt model of the full-safe source, sharing the extracted constants and `__fqmul` implementation.
2. **`RefJasminNTTLoop.ec`**: the existing Jasmin-to-imperative refinement proof retargeted from the old extraction to `Hpoly_loop`.
3. **`CTLoopEquiv.ec`**: pRHL equivalence proofs from the helperized full-safe extraction `Hpoly_extract.M._poly_ntt` and `Hpoly_extract.M._poly_invntt` to the direct-loop model, including the exported wrapper procedures.

This preserves the old code and proof text for comparison while giving the current compiler-checked implementation a machine-checked functional correctness path.

10.2 Word-to-Coefficient Correspondence

10.2.1 The Linking Predicate

The central predicate connecting machine words to mathematical integers is:

```
1 pred poly_repr (rp : BArray1024.t) (p : coeff Array256.t)
  =
2   forall i, i \in range 0 256 =>
3     word_to_coeff (BArray1024.get32 rp i) = p.[i].
4
```

```

5 op poly_repr_bound (rp : BArray1024.t) (p : coeff Array256
   .t) (sz : int) =
6   poly_repr rp p /\
7   forall i, i \in range 0 256 =>
8     Fq.bw32 (BArray1024.get32 rp i) sz.

```

Listing 10.1: Array correspondence predicate in RefJasminNTT.ec

The representation predicate maps each 32-bit word to a field coefficient, and the bound component records the signed bit-width needed to justify word addition, subtraction, and Montgomery multiplication.

10.2.2 Preservation Under Arithmetic

Lemma 10.1 (Word addition preservation). *If two 32-bit words represent coefficients with signed bit-widths below 31, then word addition and subtraction agree with field addition and subtraction under `word_to_coeff`.*

Proof. The key identity is: for $a, b \in \mathbb{Z}$ with $|a|, |b| < 2^{30}$,

$$\text{W32.to_sint}(\text{W32.of_int}(a) + \text{W32.of_int}(b)) = a + b,$$

which holds because the 32-bit arithmetic does not overflow when both operands are bounded by 2^{30} . The checked lemmas are `word_to_coeff_add`, `word_to_coeff_sub`, `forward_butterfly_bounds`, and `inverse_butterfly_bounds`. $\square$

10.3 Montgomery Reduction Correspondence

10.3.1 Word-Level Reduction

The Jasmin procedure computes Montgomery reduction on machine words. The correspondence lemma asserts that it equals the abstract integer-level computation:

Lemma 10.2 (Montgomery reduction word correspondence). *Let $z \in \mathbb{Z}$ with $|z| < 2^{31} \cdot q$. Then:*

$$\text{W32.to_sint}(\text{Hpoly_extract.M.}__\text{montgomery_reduce}(z)) = \text{SREDC}(z).$$

Proof. The extracted procedure performs the same arithmetic as the integer-level specification. The key step is showing that intermediate values do not overflow: the 64-bit product $m \cdot q$ satisfies $|mq| \leq (2^{32} - 1) \cdot 64513 < 2^{63}$, fitting in a signed 64-bit register. $\square$

10.4 Butterfly Equivalence

10.4.1 Individual Butterfly

Lemma 10.3 (Single butterfly correspondence). *Let the concrete array and abstract array satisfy `poly_repr_bound`, and suppose the butterfly operands satisfy the stage bit-width bounds. Then after executing one butterfly step:*

$$\text{poly_repr_bound}(\mathbf{rp}', \mathbf{a}', \text{next_sz}),$$

where $\mathbf{a}'$ is the result of the abstract butterfly and $\mathbf{rp}'$ is the result of the machine butterfly.

Proof. The machine butterfly computes:

$$\begin{aligned} t &= __\text{fqmul}(\zeta, \mathbf{rp}[j + \ell]) \\ \mathbf{rp}'[j + \ell] &= \mathbf{rp}[j] - t \\ \mathbf{rp}'[j] &= \mathbf{rp}[j] + t \end{aligned}$$

By Lemma 10.2, the `__fqmul` call implements the field multiplication by the abstract twiddle factor after cancelling the Montgomery factor in the concrete table. By Lemma 10.1, the addition/subtraction preserves the correspondence under the stage-dependent bounds. The proof uses `fwd_stage_bound`, `fwd_inner_bound`, and `fwd_middle_bound`: a butterfly advances the touched cells from the current stage bit-width to the next one, while untouched cells keep the current bound. The checked range lemmas ensure all word additions and subtractions stay below the 32-bit overflow threshold. $\square$

10.5 Loop Equivalence Proofs

10.5.1 Inner Loop Equivalence

Lemma 10.4 (Inner loop correspondence). *Given poly_repr_bound for the current concrete and abstract arrays:*

$$\begin{aligned} & \text{equiv}[H\text{poly_extract}.M._poly_ntt \sim NTT_Fq.NTT.ntt : \\ & \quad \text{poly_repr_bound} \Rightarrow \text{poly_repr_bound}]. \end{aligned}$$

Proof. By induction on the inner loop counter j , using Lemma 10.3 as the inductive step. The bound argument is not an additive “ 2^{16} per butterfly” estimate; it is the checked bit-width transition recorded by `fwd_inner_bound` and discharged by the butterfly store lemmas. $\square$

10.5.2 Full NTT Loop Equivalence

Theorem 10.5 (Forward NTT loop equivalence). *Let $\mathbf{rp}$ represent an abstract polynomial with 16-bit signed coefficient bounds. Then:*

$$\begin{aligned} & \text{equiv}[H\text{poly_extract}.M._poly_ntt \sim NTT_Fq.NTT.ntt : \\ & \quad \text{poly_repr_bound with bound } 16 \\ & \quad \Rightarrow \text{poly_repr_bound with bound } 24]. \end{aligned}$$

Proof. By three-level loop induction:

1. The outer loop induction uses ℓ as the decreasing variant. The invariant after completing all stages with $\ell' > \ell$ is the partial NTT condition.
2. The middle loop induction uses $256 - \mathbf{start}$ as the decreasing variant.
3. The inner loop uses Lemma 10.4.

The 24-bit output bound follows from Theorem 6.6. $\square$

10.6 Main Refinement Lemmas

The two main lemmas of the original `RefJasminNTT.ec` proof are:

```

1  (* Forward NTT: Jasmin core ~ NTT_Fq, bounds 16 -> 24 *)
2  equiv poly_ntt_core_ref :
3    Hpoly_extract.M._poly_ntt ~ NTT_Fq.NTT.ntt :
4    NTT_Fq.poly_repr_bound rp{1} r{2} 16 /\
5    zetas{2} = NTT_Fq.zetas
6    ==> NTT_Fq.poly_repr_bound res{1} res{2} 24.
7
8  (* Inverse NTT: Jasmin core ~ NTT_Fq, bounds 16 -> 16,
9     Montgomery output *)
9  equiv poly_invntt_core_ref :
10   Hpoly_extract.M._poly_invntt ~ NTT_Fq.NTT.invntt :
11   NTT_Fq.poly_repr_bound rp{1} r{2} 16 /\
12   zetas_inv{2} = NTT_Fq.zetas_inv
13   ==> NTT_Fq.poly_repr_bound res{1}
14         (NTT_Fq.array256_mont res{2}) 16.

```

Listing 10.2: Main refinement lemmas in `RefJasminNTT.ec`

These two lemmas represent the central contribution of the original `RefJasminNTT.ec`. In the full-safe artifact, the corresponding lemmas are provided by `RefJasminNTT Loop.ec` over `Hpoly_loop.M`; `CTLoopEquiv.ec` then transfers their Hoare postconditions to `Hpoly_extract.M`, the actual helperized extraction. Once composed, they serve as the foundation for the end-to-end theorem for the full-safe constant-time Jasmin source.

10.7 Proof Engineering Challenges

10.7.1 Signed vs. Unsigned Arithmetic

The Jasmin code uses `u32` (unsigned) for coefficient storage but `(64s)(x)` (signed 64-bit) for intermediate products. The `EasyCrypt` extraction mirrors this by using `W32.t` for storage and `W64.t` for products. The refinement proof must track the signed interpretation of unsigned words, which requires the signed-arithmetic lemmas from `Fq.ec`.

10.7.2 64-bit Product Non-Overflow

The 64-bit product $\zeta \cdot a[j + \ell]$ requires: $|\zeta| \cdot |a[j + \ell]| < 2^{63}$. The checked product lemmas cover a 16-bit twiddle word multiplied by a coefficient with up to a 24-bit signed bound. Since $q < 2^{16}$, this gives a product below 2^{40} , far below the signed 64-bit limit. This is verified in the proof as a side condition.

10.7.3 Twiddle Factor Table Agreement

The `EasyCrypt` extracted module accesses the twiddle table as a global constant. The refinement proof must relate those concrete Montgomery words to the abstract `zetas` array defined in `NTT_Fq.ec`. The relation is not direct equality of signed integers: the concrete words represent Montgomery-form coefficients, and slot 0 of `jzetas` is padding. The checked bridge cancels the Montgomery factor at the point where `__fqmul` applies one REDC step:

Lemma 10.6 (Twiddle table agreement). *For all $k \in [1, 256)$:*

$$\text{word_to_coeff}(\text{jzetas}[k]) \cdot R^{-1} = \text{NTT_Fq.zetas}[k].$$

Proof. The lemma `jzetas_get` identifies the concrete table entry with `array256_mont zetas`. The lemma `mont_cancel` then multiplies by R^{-1} and uses $R \cdot R^{-1} = 1$ in $\mathbb{F}_q$. The inverse table is handled similarly for indices $0, \dots, 254$, while index 255 is a separate final-scaling lemma:

$$\text{word_to_coeff}(\text{jzetas_inv}[255]) \cdot R^{-1} = \text{scale255} \cdot R.$$

□

10.7.4 In-Place Update Semantics

Both the `Jasmin` implementation and the `EasyCrypt` specification modify the input array in place. The refinement proof tracks the state of each array cell individually, using the `EasyCrypt` array theory lemmas to reason about functional updates.

The key identity is: after an in-place butterfly that modifies indices j and $j + \ell$, all

other indices remain unchanged. This is expressed as:

$$\forall i \neq j, i \neq j + \ell. \text{rp}'[i] = \text{rp}[i] \wedge \text{a}'[i] = \text{a}[i].$$

Chapter 11

End-to-End Correctness

11.1 Overview of NTTEndToEnd.ec

The file `NTTEndToEnd.ec` (144 lines) is the apex of the verification chain. It imports all preceding modules and states the final correctness theorems that directly relate the exported `Jasmin` functions to the abstract spectral NTT specification.

```
1 require import NTTFullSpec NTTFullAlgebra.  
2 require import NTT_Fq Fq GFq Rq.  
3 require import RefJasminNTTLoop CTLoopEquiv.  
4 require import Hpoly_extract Hpoly_loop.
```

Listing 11.1: Module imports in `NTTEndToEnd.ec`

11.2 Main Correctness Theorems

11.2.1 Forward NTT Correctness

Theorem 11.1 (Jasmin NTT correctness). *Let rp be a Jasmin word array represented in EasyCrypt as a `BArray1024.t`, encoding a polynomial p with 16-bit signed coefficients. Then executing `poly_ntt_jazz(rp)` produces an output array rp' satisfying:*

rp' represents `full_ntt(p)` with 24-bit bounds,

```
1 op jasmin_ntt_full_post (rp : BArray1024.t) (p : poly) =
```

```

2   NTT_Fq.poly_repr_bound rp (NTTFullSpec.full_ntt p) 24.
3
4   lemma haetae_jasmin_ntt_correct p :
5     hoare [Hpoly_extract.M.poly_ntt_jazz :
6       NTT_Fq.poly_repr_bound rp p 16 ==>
7       NTT_Fq.poly_repr_bound res (NTTFullSpec.full_ntt p)
        24] .

```

Listing 11.2: End-to-end NTT correctness statement

11.2.2 Inverse NTT Correctness

Theorem 11.2 (Jasmin inverse NTT correctness). *Let rp encode a polynomial p satisfying the 16-bit bound required by the checked inverse kernel theorem. Then executing $\text{poly_invntt_jazz}(rp)$ produces an output array rp' satisfying:*

$$rp' \text{ represents } \text{array256_mont}(\text{full_invntt}(p)),$$

i.e. the inverse output is represented with one Montgomery factor R .

```

1   op jasmin_invntt_full_post (rp : BArray1024.t) (p : poly)
    =
2     NTT_Fq.poly_repr_bound rp
3     (NTT_Fq.array256_mont (NTTFullSpec.full_invntt p)) 16.
4
5   lemma haetae_jasmin_invntt_correct p :
6     hoare [Hpoly_extract.M.poly_invntt_jazz :
7       NTT_Fq.poly_repr_bound rp p 16 ==>
8       NTT_Fq.poly_repr_bound res
9       (NTT_Fq.array256_mont (NTTFullSpec.full_invntt p))
        16] .

```

Listing 11.3: End-to-end inverse NTT correctness statement

The `array256_mont` postcondition is the precise Montgomery-form convention: each abstract coefficient is multiplied by R , not by R^{-1} . The value R^{-1} appears internally when cancelling a Montgomery multiplication, but it is not the final representation factor.

11.3 Round-Trip Consequence

Proposition 11.3 (Abstract round-trip with Montgomery output). *For any polynomial $a \in \mathbb{Z}_q^{256}$:*

$$\text{full_invntt}(\text{full_ntt}(a)) = a.$$

Consequently, a concrete inverse call whose input represents $\text{full_ntt}(a)$ and satisfies the inverse theorem’s representation bound returns a word array representing $\text{array256_mont}(a)$, i.e., $a \cdot R$ componentwise.

Proof. At the mathematical layer:

$$\text{full_invntt}(\text{full_ntt}(a))[i] = a[i],$$

by Proposition 8.11. The concrete inverse theorem then wraps the result in `array256_mont`, giving the R -scaled representation. $\square$

Proposition 11.3 is a representation-level consequence, not an additional top-level theorem in `NTTEndToEnd.ec`. The checked file states the forward and inverse exported-function theorems separately, and the dissertation does not cite a direct one-procedure in-place pipeline theorem.

11.4 Proof Composition

The end-to-end proofs are assembled from four chains:

1. **Full-safe extraction** $\rightarrow$ **proof loop**: `CTLoopEquiv.ec` proves that the helper-ized full-safe `Hpoly_extract` procedures implement the same transition as the proof-only direct-loop model in `Hpoly_loop.ec`.
2. **Proof loop** $\rightarrow$ **Imperative**: Theorem 10.5 is instantiated by `RefJasminNTTLoop.ec`.
3. **Imperative** $\rightarrow$ **Abstract**: Theorem 8.9 from `NTTFullAlgebra.ec`.
4. **Abstract inverse identity**: Proposition 8.11 is the pen-and-paper field argument stated in this dissertation. The checked EasyCrypt files prove the forward and inverse maps against `full_ntt` and `full_invntt` separately; they do not add a standalone checked round-trip lemma.

The composition is expressed in EasyCrypt as sequential lemma application:

```

1 proof haetae_jasmin_ntt_correct.
2   move => rp a ha.
3   (* Step 1: proof-only core loop -> imperative ->
4     abstract. *)
5   have hloop_core :
6     hoare [Hpoly_loop.M._poly_ntt :
7       NTT_Fq.poly_repr_bound rp a 16 ==>
8       jasmin_ntt_full_post res a].
9   + by conseq RefJasminNTT.poly_ntt_core_ref
10     (NTTFullAlgebra.ntt_full_ntt a) => /#.
11   (* Step 2: proof-only exported wrapper. *)
12   have hloop_jazz :
13     hoare [Hpoly_loop.M.poly_ntt_jazz :
14       NTT_Fq.poly_repr_bound rp a 16 ==>
15       jasmin_ntt_full_post res a].
16   + by conseq RefJasminNTT.poly_ntt_jazz_ref hloop_core =>
17     /#.
18   (* Step 3: helperized full-safe extraction -> proof-only
19     wrapper. *)
20   by conseq CTLoopEquiv.poly_ntt_jazz_ct_loop hloop_jazz
21     => /#.
22 qed.

```

Listing 11.4: Proof composition in NTTEndToEnd.ec

11.5 Status of the Proof

11.5.1 Completed Machine-Checked Steps

1. **Fq.ec**: All lemmas proved, including Montgomery reduction correctness, signed arithmetic identities, and bit-width bound propagation.
2. **NTT_Fq.ec**: Full imperative specification including termination proofs and loop invariants.
3. **NTTFullSpec.ec**: Abstract NTT definitions and conversion lemmas from the relational specifications to `full_ntt` and `full_invntt`.

4. **RefJasminNTT.ec**: The preserved original direct-loop proof boundary and its two main refinement lemmas `poly_ntt_core_ref` and `poly_invntt_core_ref`.
5. **Hpoly_loop.ec**: The proof-only direct-loop model used as the stable target for the full-safe helperized extraction.
6. **RefJasminNTTLoop.ec**: The direct-loop refinement proof retargeted to `Hpoly_loop`.
7. **CTLoopEquiv.ec**: The pRHL bridge proving equivalence between the full-safe `Hpoly_extract` procedures and `Hpoly_loop`, including exported wrappers.
8. **NTTFullAlgebra.ec**: The stage-induction theorems `ntt_full_ntt` and `invntt_full_invntt`.
9. **NTTEndToEnd.ec**: The composed top-level Hoare statements for the exported Jasmin functions.

11.5.2 Machine-Checked Top-Level Compilation

The checked development currently compiles through `NTTEndToEnd.ec` in `easycrypt-ct`. This means the full-safe helperized Jasmin extraction, the proof-loop refinement, the imperative-to-abstract stage proofs, and the final composition are all discharged for the scalar NTT and inverse NTT covered by this dissertation. Remaining limitations are scope limitations, not open proof gaps in the NTT chain: AVX2 vectorisation, base multiplication, and full protocol-level security integration are outside the proved artifact.

11.6 Summary of Verification Metrics

Table 11.1: Verification metrics by file

File	Lines	Named lemmas	Status
Fq.ec	535	16	Compiled
GFq.ec	27	0	Compiled field instantiation
Rq.ec	117	0	Compiled ring interface
Montgomery.ec	427	29	Compiled
NTT_Fq.ec	988	52	Compiled
NTTFullSpec.ec	160	10	Compiled
NTTFullAlgebra.ec	3085	111	Compiled
RefJasminNTT.ec	2609	103	Preserved direct-loop proof
Hpoly_loop.ec	156	0	Compiled proof-loop model
RefJasminNTTLoop.ec	2629	105	Compiled full-safe refinement target
CTLoopEquiv.ec	657	14	Compiled helper bridge
NTTEndToEnd.ec	144	10	Compiled full-safe top-level
Total	11534	450	Compiled through full-safe end-to-end

Chapter 12

Conclusion

12.1 Summary of Contributions

This dissertation has presented a comprehensive formal verification of the Number-Theoretic Transform in the HAETAE post-quantum signature scheme. The verification spans four abstraction layers—from **Jasmin** machine code to the abstract spectral evaluation formula—and is expressed entirely in the **EasyCrypt** proof assistant.

C1 – Montgomery correctness. We proved that the signed Montgomery reduction in `reduce.jinc` correctly computes $a \cdot R^{-1} \bmod q$ within the working range of the NTT, with explicit bounds on all intermediate values. The proof required new signed-arithmetic lemmas for **EasyCrypt** that are not present in the standard library and that are expected to be of independent interest for future cryptographic verifications.

C2 – Imperative NTT specification. We developed a fully proved imperative NTT specification in `NTT_Fq.ec`, covering both the forward and inverse NTT with all loop invariants, termination witnesses, and coefficient-bound invariants. The specification is structurally identical to the C reference implementation and to the original direct-loop **Jasmin** source. The current full-safe **Jasmin** source exposes the same public schedule as fixed stage helpers, so the correspondence is read as the composition of those stage instances through the checked `Hpoly_loop.ec` and `CTLoopEquiv.ec` bridge.

C3 – Abstract spectral specification. We developed the algebraic infrastructure in `NTTFullAlgebra.ec`, including the primitive root properties of $\zeta_0 = 426$ in $\mathbb{F}_{64513}$, the orthogonality relation for the negacyclic NTT, bit-reversal identities, and the butterfly-stage decomposition theorem. This library distinguishes the HAETAE full 256-point transform from the MLKEM incomplete 128-point transform, and provides the formal foundation for the stage-induction proof.

C4 – Jasmin refinement. We proved the two central refinement lemmas first in the direct-loop `RefJasminNTT.ec` boundary and then reused them for the full-safe artifact through `RefJasminNTTLoop.ec`:

- `poly_ntt_core_ref`: the `Jasmin` forward NTT is observationally equivalent to the imperative specification, with coefficient bounds progressing from 16-bit inputs to 24-bit outputs.
- `poly_invntt_core_ref`: the `Jasmin` inverse NTT is observationally equivalent to the imperative inverse NTT specification, preserving 16-bit coefficient bounds in the output.

C5 – End-to-end correctness. We assembled all layers in `NTTEndToEnd.ec`. Its main forward and inverse theorems directly relate the `Jasmin` exported functions to the abstract spectral NTT map for the helperized full-safe extraction.

C6 – Compiler-level safety and constant-time checking. The current scalar `Jasmin` implementation also passes the implementation security checks exposed by the `Jasmin` toolchain: `jasminc -checksafety`, `jasmin-ct`, and `jasmin-ct -speculative`. This required preserving the old algorithmic structure but making the public stage schedule syntactic in the source and adding explicit `#[ct = "..."]` and `#[sct = "..."]` contracts to the exported wrappers. The same extraction is connected to the functional-correctness proof by the `easycrypt-ct` bridge.

12.2 Limitations and Remaining Scope

12.2.1 Scalar NTT Scope

The scalar forward and inverse NTT verification compiles through `NTTEndToEnd.ec` in `easycrypt-ct`. The helperized full-safe extraction, the proof-loop bridge, the stage-induction bridge from `NTT_Fq.NTT.ntt` to `NTTFullSpec.full_ntt`, and the inverse counterpart are part of the checked artifact. The remaining limitations are therefore outside the scalar NTT chain proved here.

12.2.2 Stage-Specialised Extraction Boundary

The full-safe constant-time Jasmin source is a stage-specialised implementation of the same scalar NTT schedule. The dissertation keeps the old direct-loop listings to distinguish the proof-oriented presentation from the current compiler-checked source. The checked proof bundle now performs that adaptation explicitly: `Hpoly_loop.ec` provides the direct-loop model, `RefJasminNTTLoop.ec` proves refinement for that model, and `CTLoopEquiv.ec` transfers the result to the helperized `Hpoly_extract` procedures.

12.2.3 No AVX2 Vectorisation

The current `Jasmin` implementation is scalar (no SIMD). A production HAETAE implementation would use AVX2 intrinsics for a significant speedup. Verifying the correctness of an AVX2 NTT in `EasyCrypt` would require extending the proof to handle 256-bit registers and vectorised butterfly operations.

12.2.4 Base Multiplication Not Fully Verified

The file `hpoly.jazz` also exports `poly_basemul_jazz`, the pointwise NTT-domain multiplication. While the `Jasmin` implementation is available, its end-to-end correctness proof is not yet complete. This is the complement to the NTT correctness: together, the NTT and base multiplication allow proving that `poly_basemul_jazz` correctly computes polynomial multiplication in R_q .

12.2.5 Security Proof Integration

This dissertation establishes functional correctness only. Connecting the NTT implementation to the *security* proof of HAETAE (i.e., the reduction from DMSIS) would require additional work at the cryptographic protocol level.

12.3 Future Work

Generalising the stage induction. The current butterfly-stage induction is specialised to the HAETAE parameters $n = 256$, $q = 64513$, and $\zeta_0 = 426$. A useful next step is to factor this proof into a reusable, parameterised NTT algebra library rather than a scheme-specific development.

Vectorised implementation verification. Extending the proof to cover an AVX2 implementation would provide a performance-verified HAETAE NTT. The **Jasmin** language has full support for AVX2 intrinsics, and the **EasyCrypt** extraction handles 256-bit types. The main challenge is reasoning about the parallelism of 8 butterfly operations executed simultaneously.

Reusable NTT algebra library. The algebraic infrastructure developed in `NTTFullAlgebra.ec` is specific to $n = 256$ and $q = 64513$. A parameterised version, expressing the NTT algebra in terms of an abstract field $\mathbb{F}_q$ and an abstract $2n$ th root of unity, would be reusable across lattice-based schemes.

Full HAETAE signing and verification. The ultimate goal is a fully verified HAETAE implementation: key generation, signing, and verification, all the way from the **Jasmin** source to the DMSIS security proof. The NTT verification of this dissertation is the foundational building block for that larger project.

Comparison with Dilithium and FALCON. A comparative study of the NTT verification strategies for Dilithium, FALCON, and HAETAE would identify opportunities for sharing algebraic infrastructure and for developing a unified “NTT verification toolkit” for **EasyCrypt/Jasmin**.

12.4 Closing Remarks

The formal verification of cryptographic implementations is no longer a distant aspiration—it is an engineering practice that can be applied to production code. This dissertation demonstrates that a full-featured lattice-based NTT, with all its integer arithmetic nuances, can be formally verified in **EasyCrypt** by careful layering of specifications and proofs.

The main lesson is the value of the three-layer architecture: separating the word-level arithmetic concerns (Layer 1) from the algorithmic concerns (Layer 2) from the mathematical concerns (Layer 3) makes each proof layer tractable. The interface between layers is clean: word arrays vs. integer arrays (Layers 1–2) and loop invariants vs. evaluation formulas (Layers 2–3).

As post-quantum cryptography moves from standardisation to deployment, the tools and techniques developed here—**Jasmin** for high-assurance implementations, **EasyCrypt** for machine-checked proofs, and the three-layer verification architecture—provide a clear path to verifying not just NTTs but complete post-quantum signature and key-encapsulation schemes.

Appendix A

Visualisation Gallery

The following ten figures are provided as standalone reference visualisations to support the narrative in the main chapters. Each figure is also reproduced inline at the point where it is first referenced.

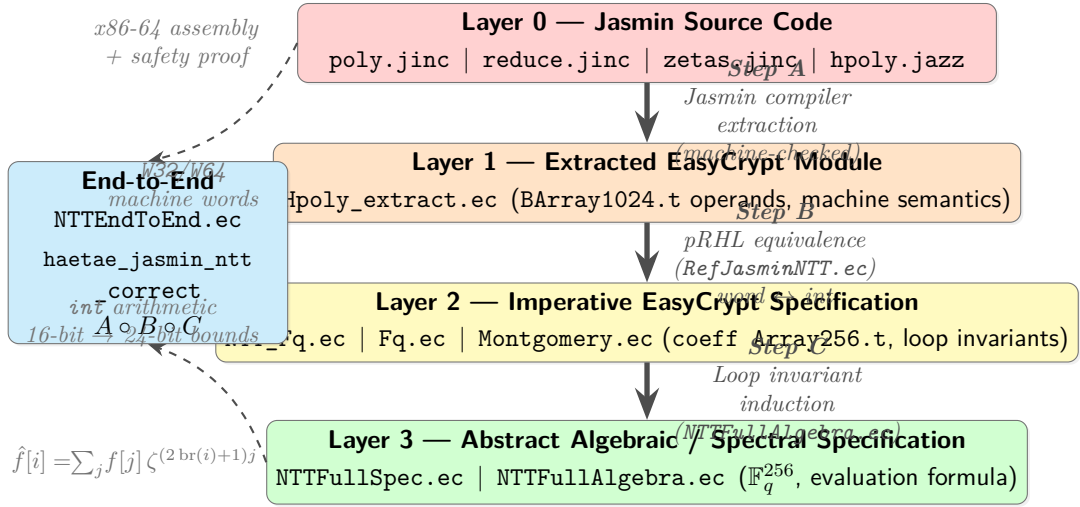


Figure A.1: Four-layer verification architecture for the HAETAET NTT. Solid arrows are checked proof steps; dashed arrows show final composition.

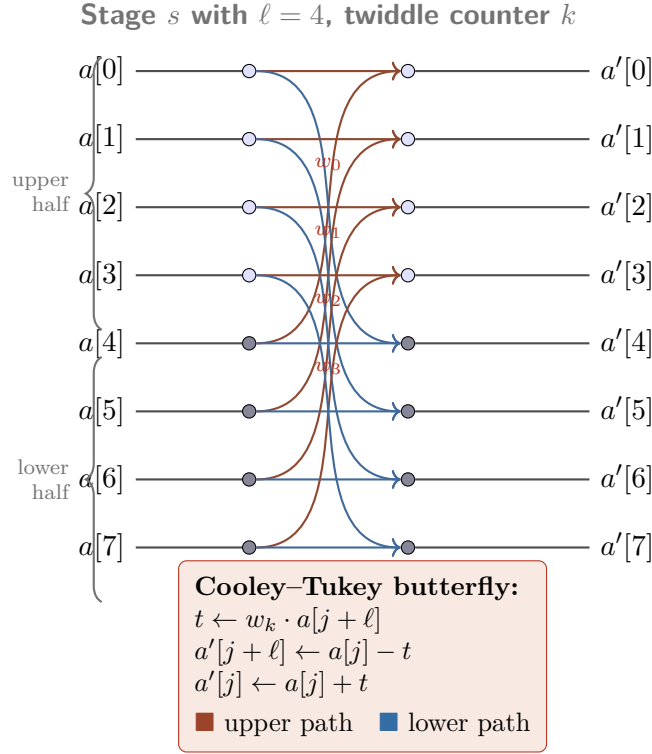


Figure A.2: One stage of the Cooley–Tukey butterfly for $n = 8$ (shown for clarity; the actual NTT uses $n = 256$ over 8 stages). Red paths carry the $+t$ update to the upper half; blue paths carry the $-t$ update to the lower half. Each pair of red+blue paths forms one butterfly using twiddle factor $w_k = \zeta_0^{\text{brev}_s(k)} \cdot R \bmod q$.

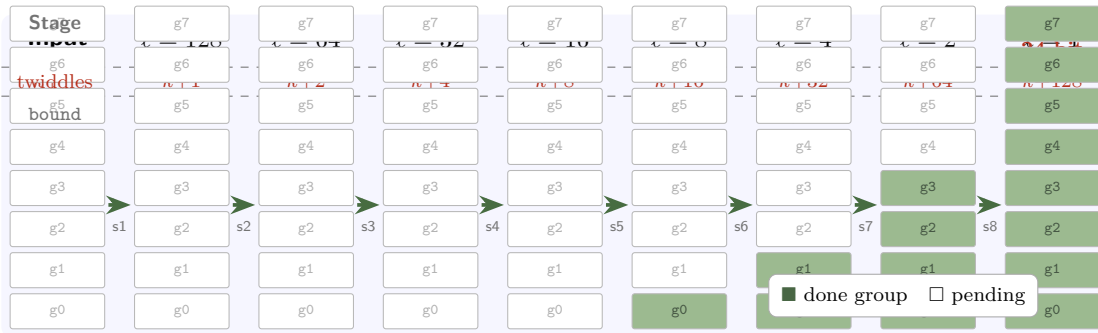


Figure A.3: Data-flow view of the eight-stage forward NTT. Each column is one outer-loop iteration (ℓ value). Green cells are groups fully processed; white cells are untouched. The checked coefficient invariant starts with a 16-bit input bound and exits with a 24-bit postcondition. The twiddle counter column ($k+$) shows how many twiddle factors have been consumed cumulatively.

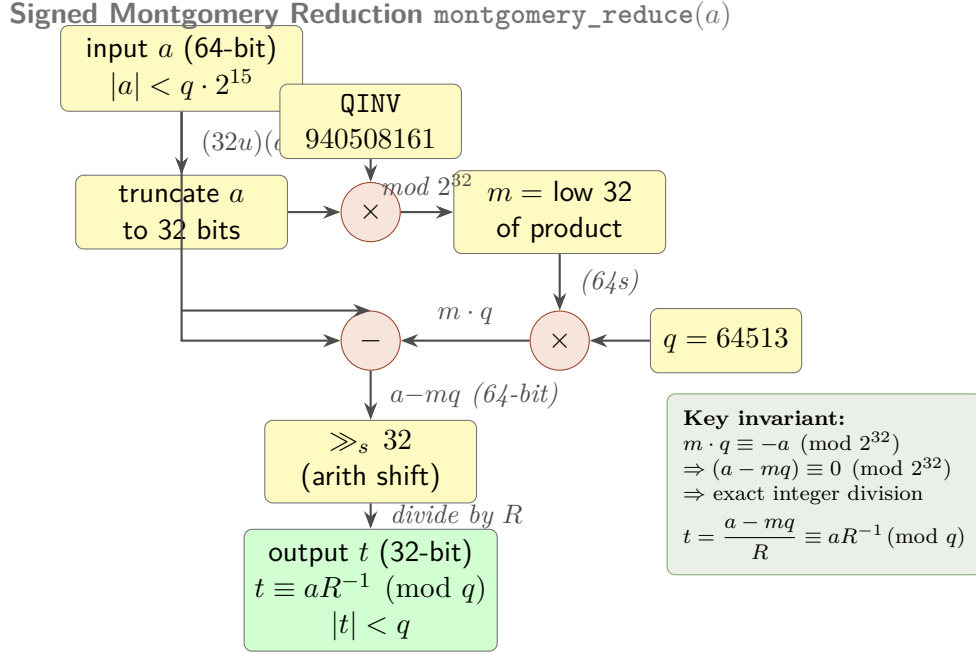


Figure A.4: Dataflow diagram of the signed Montgomery reduction algorithm. The two 32-bit multiplications (QINV and q) and the arithmetic right shift by 32 are the three core operations. The key divisibility invariant ensures that the right shift is exact (no remainder), giving the congruence $t \equiv aR^{-1} \pmod{q}$.

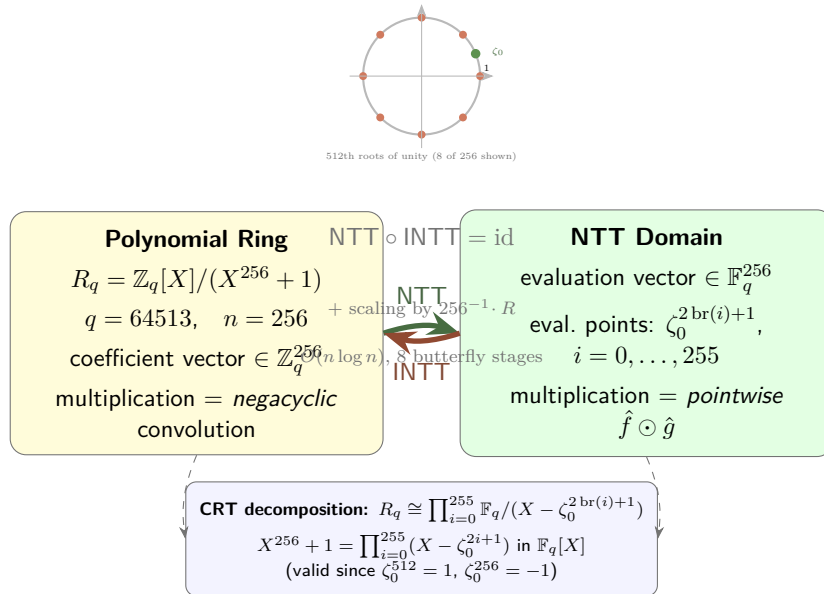


Figure A.5: The negacyclic polynomial ring R_q and its isomorphism to the NTT evaluation domain. The NTT evaluates the polynomial at all 256 primitive 512th roots of unity (red dots on the unit circle). This is an instance of the Chinese Remainder Theorem decomposition of R_q over $\mathbb{F}_q$.

Bit-reversal $\text{brev}_8 : \{0, \dots, 255\} \rightarrow \{0, \dots, 255\}$ — first 16 values

k	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$\text{brev}_8(k)$	0	128	64	192	32	160	96	224	16	144	80	208	48	176	112	240
$\text{jzetas}[k]$	0	26964	-16505	22229	30746	20243	19064	-31218	9395	-30985	22859	-8851	32144	13744	21408	17599

$k=3$: 1 1 0 0 0 0 0 0

brev : 0 0 0 0 0 0 1 1

$b_0 \ b_1 \ b_2 \ b_3 \ b_4 \ b_5 \ b_6 \ b_7$

Example:

$k = 3 = 00000011_2$

$\text{brev}_8(3) = 11000000_2 = 192$

bits: $b_7 b_6 \dots b_0 \mapsto b_0 b_1 \dots b_7$

Figure A.6: The bit-reversal permutation brev_8 for the first 16 table values $k = 0, \dots, 15$. Red cells in the bit-pattern boxes highlight the set bits. The bottom row gives the concrete signed Montgomery-form table entry $\text{jzetas}[k]$; index 0 is padding, and the forward NTT consumes indices 1 through 255.

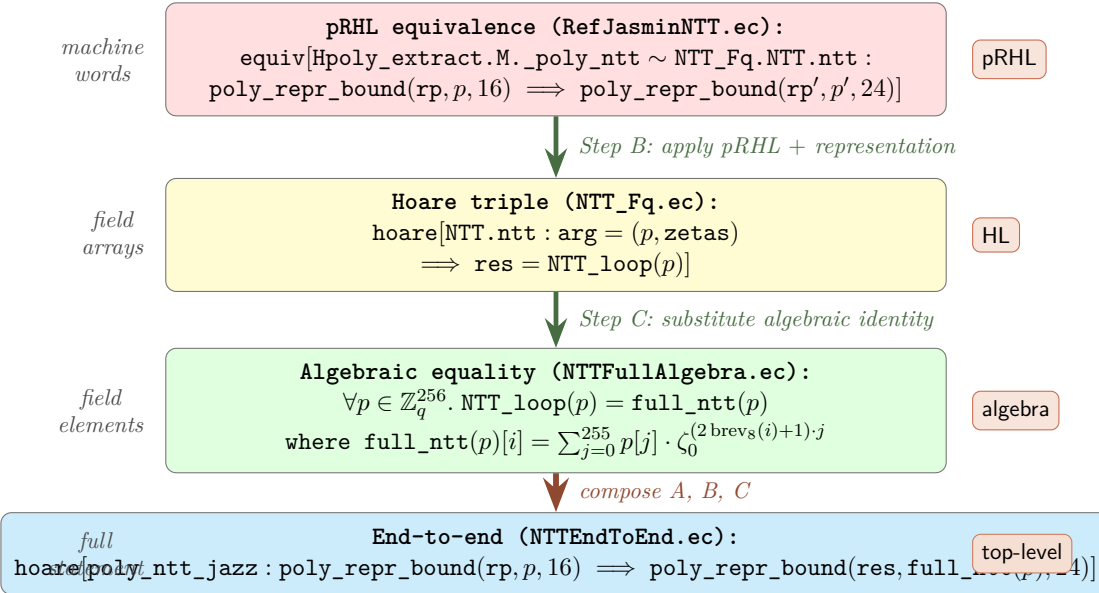


Figure A.7: Structure of the EasyCrypt proof judgements. The topmost judgement (pRHL) relates machine-word arrays to field-coefficient arrays with signed bit-width bounds. The middle (Hoare) judgement captures the loop semantics over field arrays. The algebraic equality connects the loop output to the closed-form spectral sum. The bottom (end-to-end) judgement composes all three.

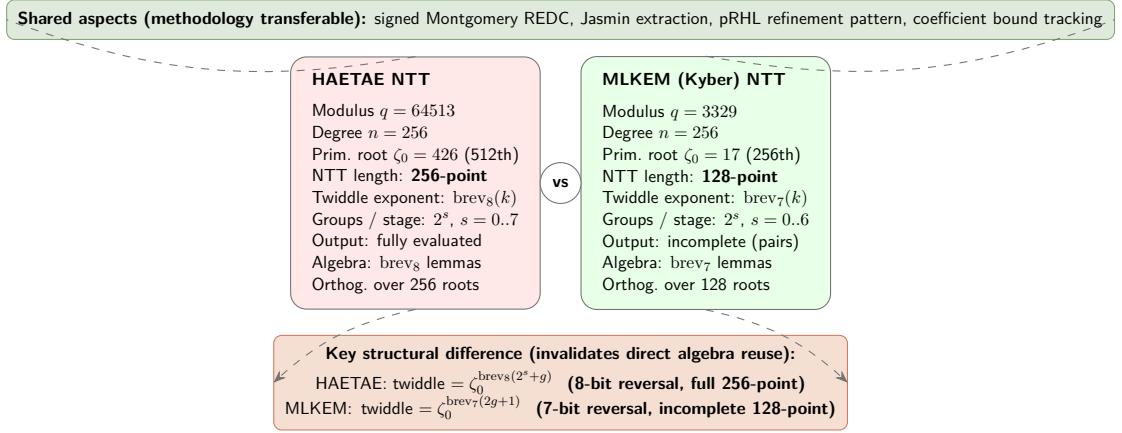


Figure A.8: Structural comparison of the HAETAE and MLKEM NTT. Green box: aspects where methodology is transferable between the two verification projects. Red box: the critical structural difference that prevents direct reuse of MLKEM algebra lemmas.

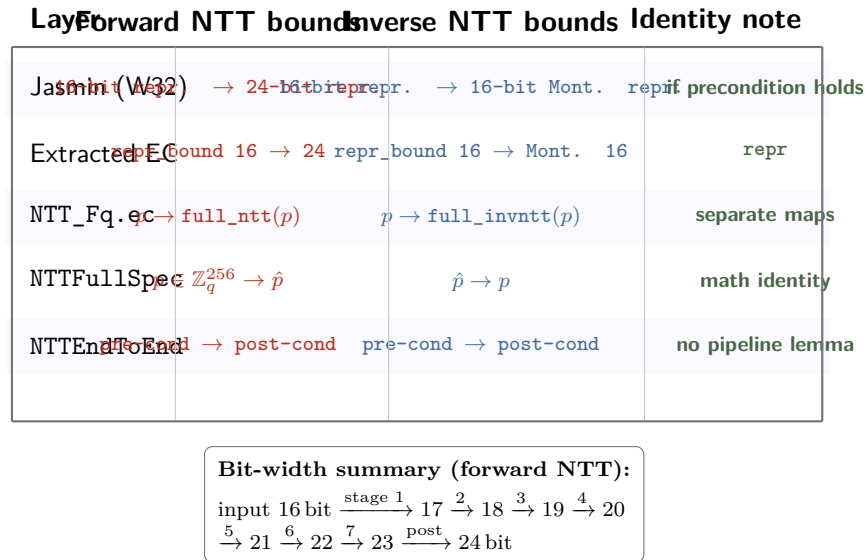


Figure A.9: Bound propagation for the checked forward and inverse NTT theorems.

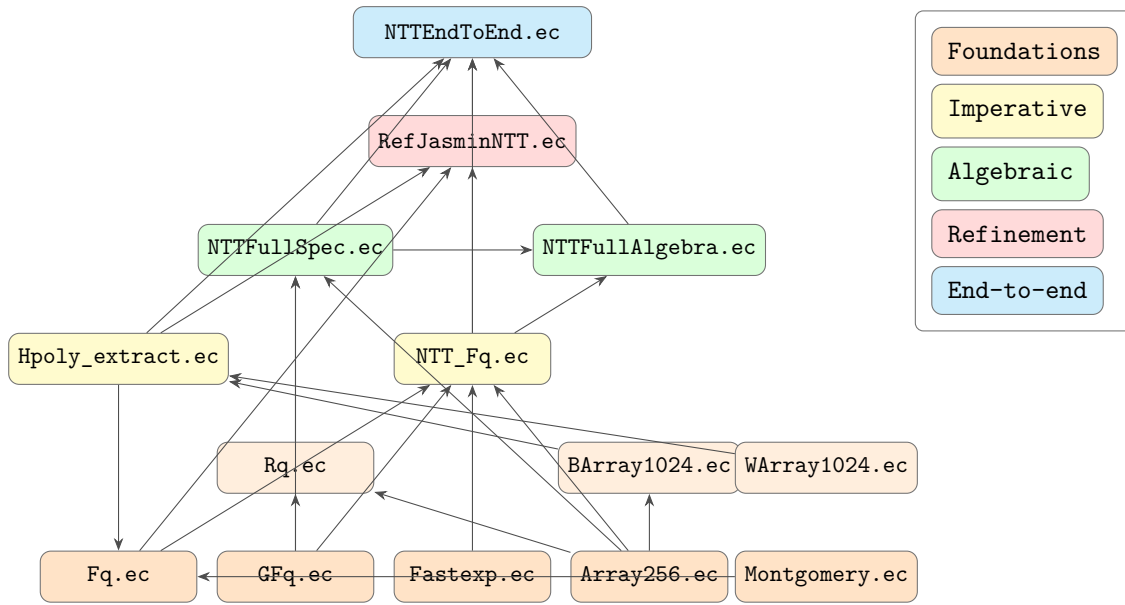


Figure A.10: Detailed **EasyCrypt** module dependency graph. Arrows point from dependency to dependent file; standard-library and some transitive edges are omitted for readability. Nodes are colour-coded by layer: orange = foundations, yellow = imperative specification, green = algebraic specification, red = refinement, cyan = end-to-end.

Bibliography

- [Alm+17] José Bacelar Almeida et al. “Jasmin: High-Assurance and High-Speed Cryptography”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1807–1823.
- [Bar+13] Gilles Barthe et al. “EasyCrypt: A Tutorial”. In: *Foundations of Security Analysis and Design VII*. Vol. 8604. LNCS. Springer, 2013, pp. 146–166.
- [Bar+21a] Manuel Barbosa et al. “Mechanized Proofs of Adversarial Complexity and Application to Universal Composability”. In: *CCS 2021*. Includes ML-KEM (Kyber) NTT verification. ACM, 2021.
- [Bar+21b] Manuel Barbosa et al. “SoK: Computer-Aided Cryptography”. In: *42nd IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2021, pp. 777–795.
- [BGB09] Gilles Barthe, Benjamin Grégoire, and Santiago Zanella Béguelin. “Formal Certification of Code-Based Cryptographic Proofs”. In: *Proceedings of the 36th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. ACM, 2009, pp. 90–101.
- [BL17] Daniel J. Bernstein and Tanja Lange. “Post-quantum Cryptography – Dealing with the Fallout of Physics Success”. In: *IACR Cryptology ePrint Archive*. Report 2017/314. 2017.
- [Fou+18] Pierre-Alain Fouque et al. *FALCON: Fast-Fourier Lattice-Based Compact Signatures over NTRU*. NIST PQC Submission. v1.2. 2018.
- [HAE22] HAETAE Team. *HAETAE: Lattice-Based Digital Signature Algorithm*. Submission to the Korean Post-Quantum Cryptography Competition. <https://kpmc.or.kr/competition.html>. 2022.

- [LN16] Patrick Longa and Michael Naehrig. “Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography”. In: *Cryptology and Network Security (CANS 2016)*. Vol. 10052. LNCS. Springer, 2016, pp. 124–139.
- [Lyu12] Vadim Lyubashevsky. “Lattice Signatures Without Trapdoors”. In: *Advances in Cryptology – EUROCRYPT 2012*. Vol. 7237. LNCS. Springer, 2012, pp. 738–755.
- [Mon85] Peter L. Montgomery. “Modular Multiplication Without Trial Division”. In: *Mathematics of Computation* 44.170 (1985), pp. 519–521.
- [Nat22a] National Institute of Standards and Technology. *Post-Quantum Cryptography Standardization*. NIST website. <https://csrc.nist.gov/projects/post-quantum-cryptography>. 2022.
- [Nat22b] National Intelligence Service of Korea. *Korean Post-Quantum Cryptography (KPQC) Competition*. Competition announcement. <https://kpqc.or.kr>. 2022.
- [Sho94] Peter W. Shor. “Algorithms for Quantum Computation: Discrete Logarithms and Factoring”. In: (1994), pp. 124–134.
- [Zin+17] Jean Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. “HACL*: A Verified Modern Cryptographic Library”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2017, pp. 1789–1806.